

DIFFERENTIABLE JERK-CONSTRAINED OPTIMAL-TIME TRAJECTORY PLANNING FOR ROBOTIC SYSTEMS

by

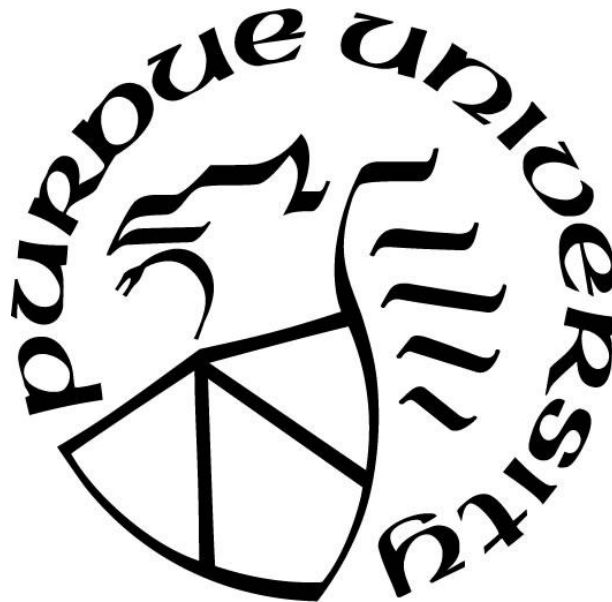
Tanyaradzwa Chinyai

A Thesis

Submitted to the Faculty of Purdue University

In Partial Fulfillment of the Requirements for the degree of

Master of Science in Electrical and Computer Engineering



Department of Electrical and Computer Engineering

Hammond, Indiana

May 2026

THE PURDUE UNIVERSITY GRADUATE SCHOOL
STATEMENT OF COMMITTEE APPROVAL

Dr. Lizhe Tan, Chair

Department of Electrical and Computer Engineering

Dr. Quamar Niyaz

Department of Electrical and Computer Engineering

Dr. Mohsen Saffari

Department of Electrical and Computer Engineering

Approved by:

Dr. Quamar Niyaz

ACKNOWLEDGMENTS

This thesis would not have been possible without the guidance and support of my advisor, Professor Lizhe Tan, whose mentorship has profoundly shaped my academic journey. I also extend my sincere thanks to Professor Quamar Niyaz and Professor Mohsen Saffari for serving on my committee and providing invaluable feedback.

To my parents, Gondai and Rosalia, thank you for instilling in me the value of challenging work and for supporting me every step of the way. I am also deeply thankful for the encouragement of my brother Tamuka, my sisters Ruvimbo, Mufaro and Tayanana.

And to my friends and colleagues at Purdue University Northwest for making this experience so rewarding.

TABLE OF CONTENTS

LIST OF TABLES	7
LIST OF FIGURES	8
ABSTRACT	9
1. INTRODUCTION	10
1.1 Background and Motivation	10
1.2 Literature Review.....	11
Spline-based and piecewise polynomial methods.....	12
Jerk minimization and smooth Trajectories	12
Industrial Robotics and Human–Robot Applications.....	14
1.3 Research Objectives and Methodology	15
1.4 Thesis Structure	16
2. KINEMATIC AND DYNAMIC MODELING OF MANIPULATORS	17
2.1 Forward and Inverse Kinematics	17
2.2 Optimization Approaches in Kinematic Analysis.....	24
2.2.1 Inverse Kinematics as an Optimization Problem.....	25
2.2.2 Handling Redundancy and Constraints	25
2.2.3 Meta-Heuristic Algorithms (PSO and GA)	26
2.2.4 Multi-Objective Frameworks.....	26
2.3 The Discrepancy between Kinematic Planning and Dynamic Reality:	27
Variable inertia.....	27
Torque saturation and tracking deviations.....	27
3. TIME OPTIMAL TRAJECTORY PLANNING AND JERK SMOOTHING.....	29
3.1 Overview of Trajectory Planning Challenges.....	29
3.1.1 Classical Approaches to Trajectory Planning.....	29
3.1.2 Polynomial-Based Trajectories.....	29
The Cubic Polynomial (Degree 3)	30
The Quintic Polynomial (Degree 5)	30
Limitations Regarding Optimality and Constraints	31
3.1.3 B-Splines and NURBs Representation	31

Mathematical Framework and Weights	32
Local Support and the Strong Convex Hull Property	33
Limitations in Time-Optimal Planning	33
3.2 Development of 6-7-6 Polynomial with Differentiable Jerk	34
3.2.1 Polynomial Structure	34
3.2.2 Boundary and Continuity Conditions	35
3.2.3 Computational Implementation	36
3.3 Algorithm Design and Implementation	36
3.3.1 Weighted Multi-Objective Particle Swarm Optimization (W-MOPSO).....	37
3.3.2 Time-Scaling Integration	39
3.4 Experimental Validation and Results.....	39
3.4.1 Kinematic Profile Validation	39
3.4.2 Simulation Verification	44
3.4.3 Statistical Performance of the 6-7-6 Polynomial Trajectory	45
4. OPTIMALITY ALGORITHMS AND ADVANCED JERK SMOOTHING TECHNIQUES.	
.....	47
4.1 Overview.....	47
4.2 S-Curves Trajectory Model.....	47
Strengths of the S-curve	48
4.3 Error Function (erf) – Based S-Curve Formulation.....	49
4.3.1 Mathematical derivation of the Gaussian Jerk.....	49
4.3.2 Optimization via Weighted Multi-Objective Particle Swarm Optimization (W-MOPSO)	
.....	52
Particle Representation and Search Space	53
The Multi-Objective Cost Formulation.....	53
Kinematic and Spatial Constraint Handling.....	54
Swarm Convergence	54
4.3.3 Implementation	55
4.4 Algorithm Design and Implementation	56
4.5 Experimental Validation and Results.....	58
4.5.1 Spatial Tracking and Path Fidelity	58

4.5.2	Simulation verification	59
4.5.3	Statistical Performance of the Multi-Waypoint Gaussian S-Curve	61
4.5.4	Kinematic Profile Validation (The Gaussian Advantage)	62
5.	CONCLUSION AND FUTURE WORK	65
5.1	Summary of Contributions.....	65
5.2	Key Findings.....	66
5.3	Future Work.....	66
	REFERENCES	68
	PUBLICATIONS.....	72

LIST OF TABLES

Table 2.1 DH Parameters for MyCobot 280 Pi.....	20
Table 3.1 Waypoints Chosen for Different Configurations in the Experiment.	39
Table 3.2 End Effector Path follower Graphs for Paths 2 to 5	41
Table 3.3 Kinematic analysis of the 676 joint motion.	43
Table 3.4 Robotic Visualization of Path 1 to Path 5 6-7-6 trajectories	44
Table 3.5 Multi-Run Performance Metrics for the 6-7-6 Polynomial Trajectory.....	46
Table 4.1 Algorithm for WMOPSO for Path Optimization (Pseudo Code)	55
Table 4.2 End effector follower path for Paths 2 to 5.....	59
Table 4.3 Robotic Visualization of Path 1 to Path 5 Gaussian Trajectories.	60
Table 4.4 Multi-Run Performance Metrics for the Gaussian Trajectory Planner.....	61
Table 4.5 Kinematic profiles for Paths 2 to 5	64

LIST OF FIGURES

Figure 2.1 Cartesian coordinate system and DH parameters for a 6 DOF robotic manipulator...	19
Figure 3.1 WMPSO Flowchart	38
Figure 3.2 Path 1 Main profile analysis of the end-effector trajectory of the 6-7-6 trajectory planner.	40
Figure 3.3 Joint space trajectory profiles demonstrating C^4 continuity generated by the 6-7-6 polynomial formulation	42
Figure 3.4 6-7-6 Path 1 (a) and Path 4 (b) Robot verification visual.....	45
Figure 4.1 Demonstration of how a low σ mimics the Dirac delta function (a massive, instantaneous spike in jerk), while the high σ flattens out into a gentle curve.....	50
Figure 4.2 Impact of the standard deviation on error function-based acceleration profiles.	50
Figure 4.3 Illustration of the Gaussian curve jerk, acceleration, velocity and position intervals.	52
Figure 4.4 Flowchart of WMPSO applied to the gaussian trajectory planne	57
Figure 4.5 Main profile analysis of end effector trajectory of the Gaussian Bell curve.	58
Figure 4.6 Gaussian Path 1(a) and Path 4(b) Robot verification visual.....	60
Figure 4.7 Kinematic Profile of individual joint motions for Gaussian S-Curve.	63

ABSTRACT

Time-optimal trajectory planning for industrial manipulators often drives actuators to their physical limits, causing high-frequency vibrations and torque saturation due to discontinuous or unconstrained jerk. Conventional strategies, such as the adoption of global polynomial and piecewise spline function for trajectory planning, often fail to achieve simultaneous spatial accuracy and transient efficiency while meeting the kinematic constraints. This thesis introduces two new trajectory planning systems that use a weighted multi-objective particle swarm optimisation (W-MOPSO) algorithm to balance execution speed and mechanical longevity. The first system uses a piecewise 6-7-6 polynomial formulation for trajectory planning, which initiates a 22-variable linear equation system, ensuring the C4 (4th-order) kinematic continuity and a differentiable jerk. The second algorithm, which is inspired by the Gaussian bell curve, produces an erf-function-based S-curve by arranging the jerk profile as a sequence of smooth and parameterised Gaussian pulses. Furthermore, a 6-DOF manipulator is simulated and validated using the real robotic model (Elephant Robotics myCobot 280 Pi operator). The developed W-MOPSO algorithm accounts for the execution time and imposes hard penalties for Cartesian position and orientation errors. Experimental results show that the proposed method effectively eliminates the jerk spikes. The erf-function-based planner provides a robust, time-optimal solution for real-time control, delivering smooth, precise, and vibration-free motion in complex robotic applications.

1. INTRODUCTION

1.1 Background and Motivation

The development and refinement of automated systems represent an extensive process, coming together over many repetitions. A peek into history shows, many devices have been constructed utilizing the materials and intellectual resources available within their respective eras. The evolution of the Antikythera mechanism, considered to be the oldest analog computer, to the current high precision robotic manipulator shows the usefulness of automation. Historically, industrial manipulators were primarily deployed for repetitive tasks lacking high-precision requirements [1]

The development of industrial robotic operators is parallel to the introduction of Unimate in 1961. The Unimate was an electromechanical device, equipped with a hydraulic arm, designed to carry out a number of operations, such as material handling and spot welding, at the assembly line [2]. As from its origin, the industry has undergone significant improvements in robotics standards and operation owing to mandatory technological developments. These include improved direct frameworks, more precise actuators and input devices, increased flexibility, and more widespread use of training procedures that independently comply with their surroundings, together with progress in planning, systematic planning, and gesture planning.

This thesis focuses on path or trajectory planning, which is related to the guidance of the manipulator along a chosen pathway. In order to ensure smooth, energy-efficient, and durable motion, there are requirements for selecting the appropriate speed and acceleration trajectories [3]. The techniques that have been used can be grouped into five distinct categories based on their time of inception, the utility, strength, and weaknesses of the techniques. These categories are:

- Classical trajectory planning – includes point to point, joint/Cartesian interpolation, polynomial, trapezoidal and S- curves.
- Optimization-based trajectory planning – these are heavily used for precision tasks and incorporate time-optimization techniques and energy-optimization to achieve optimal control. The optimal control represents the best compromise to be made within the different opposing constraints of the motion.

- Geometric and kinematic trajectory planning – they are used for obstacle avoidance, including C-space, potential fields and via-points.
- Modern AI-based trajectory planning – these are especially useful in ever-changing and stochastic environments like human-robot interaction, unmanned rovers, or any application of high-level autonomy. Techniques include the use of deep neural networks for reinforcement learning and imitation learning.

1.2 Literature Review

Traditional robotic trajectory planning often relies on simple point-to-point motion with a basic speed profile. The trapezoidal speed profile, where the automaton accelerates at a steady rate to reach cruising speed, maintains that speed, and then gradually decelerates to terminate. This “bang-bang” tactic minimizes travel time within the limits of the acceleration limit but nevertheless causes abrupt changes in acceleration by waypoints, causing an instantaneous, infinite jerk or mechanical jolt. To cope with this particular issue, the S-curve profile introduces more transitions alongside gradually increasing acceleration, or else, in the requirements of the third derivative, the constant jerk — establishing smooth passage [4] In particular, the S-curves distribute the acceleration adaptively over the duration to maintain the jerk within the desired bounds [4]. The decrease in jerk contributes to reducing vibration and promoting safer and more stable gestures [5].

The trajectory planner also uses polynomial interpolation between the waypoints. The simple scenario is a single part movement; a cubic or quintic polynomial is therefore appropriate to this situation. Speed and, if desired, acceleration starts and finishes at the specified values. For fundamental moves and higher-order polynomials (5th, 7th, etc.) allow a further derived function for smoothness. For example, a cubic polynomial, together with zero start/stop and otherwise quintic polynomial, is often used for uninterrupted acceleration [6]. A recent study validates that “the cubic polynomial technique is one of the most common approaches to form a trajectory [7]” and that higher-order polynomials may better satisfy velocity/acceleration constraints [6]. Nevertheless, the pure polynomials for trajectory planning are usually restricted to single-segment (two-point) moves. Key differences of these classical profiles include:

- Trapezoidal profile: possess minimal travel time under acceleration/velocity limits, but acceleration changes instantaneously, causing jerk spikes.

- S-curve profile introduces linear acceleration ramps (constant jerk) to smooth motion, reducing vibration at the cost of slightly longer execution.
- Polynomial (e.g. cubic/quintic): directly interpolate motion with specified endpoint derivatives. For example, cubic trajectories with zero start/end velocity yield C^2 continuous moves; quintic can also fix acceleration endpoints, yielding C^2 continuity in velocity[8].

Overall, these classical methods trade off speed versus smoothness. As one study notes, minimizing time tends to push toward bang–bang control (high jerk), whereas minimizing jerk yields smoother but slower motions[9]. In practice, simple profiles like trapezoidal or low-order polynomials are easy to implement and satisfy joint limits, but they sacrifice jerk continuity[3],[9].

Spline-based and piecewise polynomial methods.

The automaton has a wider application of piecewise polynomial and spline interpolation to manage multi-segment paths and via points. Cubic spline, B-spline, non-uniform rational B-Splines (NURBS), and other fundamental functions allow the joining of a large number of waypoints with limited continuity.

Gasparetto and Zanotto [10] investigated the trajectories, which are mainly constructed by polynomial, spline, Bezier curve, NURBS, and other interpolation functions in robotics to ensure acceleration or jerk continuity. In practice, the cubic B-splines have been used to design minimum-time joint trajectory, taking advantage of their smoothness [10]. Quintic polynomials are also common in piecewise shape. Spline-based methods enable the flexible creation of complex structures. The spline and piecewise polynomial methods have been the main path planning method of robotic trajectory, providing precise continuity directly and the ability to adapt any shape with built-in smoothness.

Jerk minimization and smooth Trajectories

The recent investigation is increasingly focused on jerk reduction and ensuring a smooth, differentiated gesture. Jerk, derivative of acceleration, directly influences robot vibration, wear, and activity. Restricting jerk output provides a smoother trajectory and produces less mechanical stress. For example, Lee et al. [11] proposed a jerk-constrained trajectory, for the boosted energy

efficiency, longevity, and safety by avoiding sudden acceleration jumps [10]. Without the jerk limit, a time-optimal trajectory may have a large acceleration discontinuity which stimulates vibration and increases the track error.

There are a number of approaches which directly reduce the jerk as an objective. For instance, Wu and Zhang [12] describe a quadratic optimization method to minimize the end-effector jerk of a robotic arm using a polynomial curve to reach real-time organizing demand; their experiment demonstrates imperative smoothing compared to unoptimized alternatively NURBS-based trajectory [12], [13]. Similarly, another source [9] used a 7th-degree B-spline whose speed, acceleration, and jerk are all continuous, and improved part intervals to achieve minimal span issues and to maintain the current smoothness.

An alternative method explicitly restricts jerk in the optimization proposed in the literature [11] to obtain a time-optimal joint trajectory involving a piecewise cubic spline together with enforced C3 continuity (incessant jerk) between portions. They transform the problem into second-order cone programs to manage the nonlinear jerk constraints, jointly optimizing both time and jerk. The results indicate that jerk restraints contribute to robot productivity and trajectory smoothness compared to unconstrained solutions.

In summary, a contemporary planner uses a higher-order spline or precise jerk restraint to produce a different jerk profile. The goal should always be a smooth gesture to avoid sudden jerks, thereby enhancing energy usage and robot longevity. For instance, another source in the literature [9] shows that the imposition of the jerk limit significantly reduces the peak control and energy consumption of real automaton execution [10]. In the context of human-robot interaction, jerk continuity is even more important: planners frequently use a spline to ensure that the derived function is continuous before acceleration, alongside confined jerk, stopping abrupt acceleration that could startle or otherwise injure a human confederate [12].

- Optimization-based and multi-objective planning: Trajectory planning is often a problem of optimization with decision-making frameworks which simultaneously optimize multiple and objectives to determine the best trade-offs between the optimal trajectories.
- Convex and sequential optimization: Planners can effectively plan for a time-optimal alternatively smooth trajectory by discretizing the path and using convex relaxation. For instance, Debrouwere et al. Moreover, different individuals have shown that trajectory limitations (including torque rate limits) can be convexified and solved by consecutive

convex scheduling, otherwise second-order cone programming (SOCP) [9-10]. Li et al. (2022) decompose time-and-jerk optimization in convex SOCP subproblems to obtain jointly time- and jerk-optimal designs. Other plants use consecutive linear/quadratic scheduling (SLP/SQP) iteratively to manage the third sequence (jerk) restriction during continuous convergence.

- Metaheuristics and evolutionary methods: The robot trajectory objective may be nonlinear or even nonconvex, so that population-based search is sought. Zhao et al. (2022) applied a hybrid whale optimization + biological algorithm (WOA-GA) to optimize joint trajectory, aiming at the entire time and the vitality of squared jerk, ensuring jerk-continuous 5th-order B-spline [14]. Likewise, sources in literature [15], [16] described a multi-objective predicament (minimize duration, jerk, and energy) for an industrial arm using the non-dominated sorting genetic Algorithm) NSGA-II to tune a 5th-order B-spline trajectory. Other researchers used particle swarm optimization (PSO), cuckoo search, or hybrid methods in similar multi-objective settings

A consolidative movement is a multi-objective optimisation, where planners commonly trade moments with smoothness or energy. For example, a source [9] reported that reducing time interval stimulates kinematic requirements, while reducing jerk smooths the motions but extends duration. Hence, many methods change the weighted sum using Pareto procedures to tackle the problem. A source in the literature [17] discusses, adding a jerk designation to the cost requires careful weight tuning, given that the solution can be very responsive to the trade-off between time and smoothness. Optimization can also extend to a wider range of motion aspects, including but not limited to energy optimization.

In summary, modern trajectory planning leverages optimization at its core. Methods range from convex programming (ensuring global convergence) to heuristic search (handling complex objectives).

Industrial Robotics and Human–Robot Applications

The above-mentioned trajectory tactics are applied to industrial operators and cooperative robots. In the case of industry, for instance, speed and precision are combined in the production line, painting, welding, and automation. Time-optimal routing maximizes throughput, while jerk-

minimized routing minimizes wear and vibration, primarily aiming at greater accuracy and lower maintenance costs [18]. Different analyses tested their algorithms on the 6 DOF arms and simulated industrial robots and reported smooth joint profiles. For instance, Gasparetto et al. explicitly targets a 6-joint industrial automaton using a B-spline/WOA-GA planner to obtain a reduced execution time and reduced jerk [9].

Jerk minimization is even more essential for safety and tranquillity in human–robot interaction (HRI) and collaborative scenarios. A smooth, predictable gesture enhances perception and prevents abrupt movements. Experts extensively use jerk-continuous splines for co-manipulation. For instance, Braglia et al. (2023) [19] encodes a trajectory with a third-order B-spline so that the speed and acceleration are constant while jerk is bounded. This ensures that if the robot must accelerate or stop abruptly in front of a human being, it does not cause impulsive acceleration. C3-continuous trajectory (continuous jerk) is highly preferred for collaborative robotics.

1.3 Research Objectives and Methodology

This thesis focuses on proposing systems which achieve a minimal trajectory duration and a smooth and differentiated jerk profile. The methodology proceeds in the following stages:

- **Trajectory Construction:** Candidate trajectories are generated using a combination of classical and novel planning approaches. Classical methods include standard polynomial or trapezoidal velocity profiles, while novel algorithms explore more advanced designs.
- **Optimization:** Each candidate trajectory is treated as an optimization problem. We apply a *weighted multi-objective Particle Swarm Optimization (W-MOPSO)* algorithm to determine a better trajectory. The algorithm plays a balancing act on objectives such as minimizing travel time, reducing jerk, and produces an optimum trajectory for each case.
- **Simulation and Implementation:** The trajectory profile solutions are simulated in MATLAB, the primary computational environment, capable of encompassing all the relevant considerations. The paths are then assessed on a 6-degree-of-freedom robotic arm (the Elephant Robotics myCobot 280 Pi) to evaluate real-world performance and feasibility.

- **Performance Analysis:** Each trajectory profile is re-evaluated through simulation. We quantify performance using metrics like total execution time and the integrated squared jerk for each trajectory. These metrics reveal the trade-offs between speed and smoothness in the generated trajectories.

1.4 Thesis Structure

This thesis is structured into five chapters, beginning with the introduction as Chapter 1. Chapter 2 explores methods for solving the inverse kinematics problem and derives the necessary formulas for the foundations of path planning. It details the forward-kinematics derivation for the robotic arm used in this study, explains how the arm is modeled and visualized in MATLAB. Lastly, it describes the forward-kinematics function used to check the inverse-kinematics results.

Chapter 3 deals with the theory and mathematical environment for systematic planning for time-optimal trajectory. It introduces the 6–7–6 polynomial, which is the one of main contributions of the thesis, discusses the obstacles to time-optimal strategies, and compares the advantages and disadvantages with classical methods. The experiment designed to evaluate the suitability and performance of the 6–7–6 polynomial is also described in this part.

Chapter 4 reviews jerk-smoothing concepts and their mathematical foundations, as well as reappraisal sector practices for producing smooth, differentiable jerks. In this chapter, an argument will be made that advocates for the practicality of S-curve techniques and proposes a new S-curve model that uses Gaussian functions to ensure third derivative continuity. The chapter concludes with experiments and results that support the proposed approach.

Finally, Chapter 5 summarizes the contributions and discusses the future research.

2. KINEMATIC AND DYNAMIC MODELING OF MANIPULATORS

This chapter provides a comprehensive review of the robot kinematics framework adopted in this study. It begins with the forward kinematics of the MyCobot 280 Pi, employing the conventional Denavit-Hartenberg (DH) methodology. The DH parameters can be used as an essential tool to precisely model the robot's geometric structure and establish its Cartesian coordinate system. Next, the chapter describes techniques used for determining inverse kinematics and emphasizes the role of the joint configuration for the desired end-effector position. At the last part of the chapter, several optimization approaches applied to trajectory planning and kinematic analysis are explored.

2.1 Forward and Inverse Kinematics

Forward kinematics refers to the process of determining the position and orientation of a robot's end effector based on its joint parameters. Given the configuration of each joint and the robot's dimensions, the forward kinematics uses geometrical techniques to create mathematical models that can compute the spatial pose of the end-effector in the established cartesian coordinate system. One common geometrical technique is the Denavit-Hartenberg convention which is a standardized method established for this purpose. It narrows down and simplifies the process of describing the spatial relationships between adjacent links of a robot by assigning coordinate frames and defining four parameters for each joint: link length, link twist, joint angle, and joint offset. With these DH parameters, the transformation matrices are established to represent the position and orientation of each link relative to the previous one. The DH method is widely used due to its systematic approach to kinematic modeling, enabling efficient computation of both the forward and the inverse kinematics [3].

Note that the inverse kinematic is the contrast to the forward kinematic. It involves determining the joint parameters required to achieve the desired position and orientation of the robot's end effector. This process is inherently more complex than forward kinematics for the following reasons:

- One too many or no solution ambiguity: For any given pose (position and orientation) of the end effector, there is likely to exist multiple joint angle combinations that satisfy it, especially in redundant manipulators which is when the DOF is more than what is needed. This ambiguity does not exist for forward kinematics. Conversely, for some target poses, no valid configuration exists because of joint limitations or singularities. The inverse kinematics (IK) must handle unreachable poses, while the forward kinematics (FK) always yields a valid pose from a given joint state.
- Nonlinearity and complexity of equations: As the DOF increases, the IK becomes highly nonlinear and consequently requires techniques that can solve these equations. Additionally, the closed form of the IK which gives us the analytic inverse solution may not exist [20].
- Certain joint configurations lead to singular Jacobian matrices (rank deficiency) where minor changes in joint angles produce no meaningful end-effector change. Near singularities, the IK algorithms can become ill-conditioned, suffer from numerical instability, or they diverge.

This thesis explores advanced aspects of robotic operation, emphasizing that forward and inverse kinematics form the foundation of trajectory planning. These principles are essential for achieving precise and reliable motion paths. Based on a comparative analysis of the manufacturer's specifications for the desktop robotic arm, myCobot 280 Pi by Elephant Robotics[14], the Cartesian coordinate system of the collaborative robotic arm is depicted in Figure 2.1.

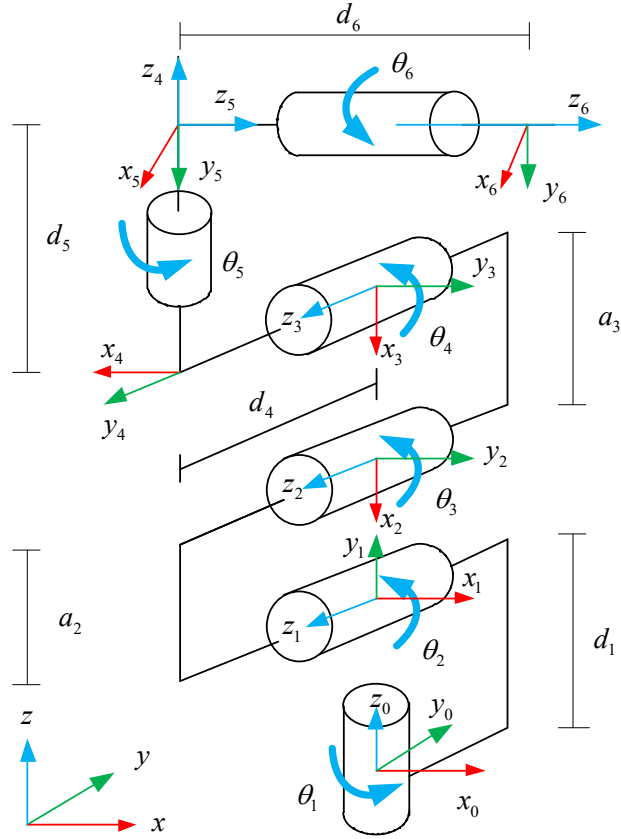


Figure 2.1 Cartesian coordinate system and DH parameters for a 6 DOF robotic manipulator.

Based on the established cartesian coordinate system in Figure 2.1, the DH parameters for the MyCobot 280 Pi[22] are formulated as shown in Table 2.1 below. It is scalable to industrial UR 3 robot [23], shown in Table 2.2 Figure 2.2 shows two robotic arms.

Table 2.1 DH Parameters for MyCobot 280 Pi

Joint #	$\Theta(rad)$	$d(mm)$	$a(mm)$	$\alpha(rad)$
0 – 1	θ_1	d_1	0	$\frac{\pi}{2}$
1 – 2	$\theta_2 - \frac{\pi}{2}$	0	$-a_2$	0
2 – 3	θ_3	0	$-a_3$	0
3 – 4	$\theta_4 - \frac{\pi}{2}$	d_4	0	$\frac{\pi}{2}$
4 – 5	$\theta_5 + \frac{\pi}{2}$	d_5	0	$-\frac{\pi}{2}$
5 – 6	θ_6	d_6	0	0

Table 2.2 Specific DH parameters of robotic arms.

Model	d1 (mm)	d4 (mm)	d5 (mm)	d6 (mm)	a2 (mm)	a3 (mm)
myCobot 280 Pi	131.56	66.39	73.18	43.6	110.4	96
UR3	151.9	112.35	85.35	81.9	243.65	213.25



a. MyCobot 280 Pi [22]



b. UR3 [23]

Figure 2.2 Robotic arms.

In optimization-based trajectory planning, one or more fitness functions are used to iteratively minimize errors. In the present work, the outputs of the optimization algorithm are the joint rotation angles of the robotic arm; therefore, it is necessary first to establish a forward-kinematic model that transforms these joint rotations into the end-effector pose so that the positional error can be quantitatively evaluated and minimized. The precision control of the robot in this study is anchored in the end-effect's position.

Accordingly, the first step is to derive the end-effector position, written in Equation (2-1), where the vectors n , o and a correspond respectively to the normal orientation, and approach axes of the end-effector. Since the z -axis coincides with the rotation axis of the wrist joint, the robot approaches the object along the gripper's z -axis, which is therefore defined as the approach (a) axis. The orientation of the gripper as it approaches the object is called the orientation (o) axis.

The x-axis, being orthogonal to both the z- and a-axes, is designated as the normal (n) axis [3].

The frame is defined in Equation (2-1)

$$F = \begin{bmatrix} n_x & o_x & a_x & p_x \\ n_y & o_y & a_y & p_y \\ n_z & o_z & a_z & p_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (2.1)$$

where the orientation and position parameters can be obtained via (2-2) to (2-19).

$$A = \begin{bmatrix} \cos \theta & -\sin \theta \cos \alpha & \sin \theta \sin \alpha & a \cos \theta \\ \sin \theta & \cos \theta \cos \alpha & -\cos \theta \sin \alpha & a \sin \theta \\ 0 & \sin \alpha & \cos \alpha & d \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (2.2)$$

$$A_1 = \begin{bmatrix} \cos \theta_1 & 0 & \sin \theta_1 & 0 \\ \sin \theta_1 & 0 & -\cos \theta_1 & 0 \\ 0 & 1 & 0 & d_1 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (2.3)$$

$$A_2 = \begin{bmatrix} \sin \theta_2 & \cos \theta_2 & 0 & -a_2 \sin \theta_2 \\ -\cos \theta_2 & \sin \theta_2 & 0 & a_2 \cos \theta_2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (2.4)$$

$$A_3 = \begin{bmatrix} \cos \theta_3 & -\sin \theta_3 & 0 & -a_3 \cos \theta_3 \\ \sin \theta_3 & \cos \theta_3 & 0 & -a_3 \sin \theta_3 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (2.5)$$

$$A_4 = \begin{bmatrix} \sin \theta_4 & 0 & -\cos \theta_4 & 0 \\ -\cos \theta_4 & 0 & -\sin \theta_4 & 0 \\ 0 & 1 & 0 & d_4 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (2.6)$$

$$A_5 = \begin{bmatrix} -\sin \theta_5 & 0 & -\cos \theta_5 & 0 \\ \cos \theta_5 & 0 & -\sin \theta_5 & 0 \\ 0 & -1 & 0 & d_5 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (2.7)$$

$$A_6 = \begin{bmatrix} \cos \theta_6 & -\sin \theta_6 & 0 & 0 \\ \sin \theta_6 & \cos \theta_6 & 0 & 0 \\ 0 & 0 & 1 & d_6 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (2.8)$$

$$\begin{aligned} {}^0T_6 &= A_1 \times A_2 \times A_3 \times A_4 \times A_5 \times A_6 \quad (2.9) \\ &= \begin{bmatrix} \cos \theta_1 & 0 & \sin \theta_1 & 0 \\ \sin \theta_1 & 0 & -\cos \theta_1 & 0 \\ 0 & 1 & 0 & d_1 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\ &\times \begin{bmatrix} \sin \theta_2 & \cos \theta_2 & 0 & -a_2 \sin \theta_2 \\ -\cos \theta_2 & \sin \theta_2 & 0 & a_2 \cos \theta_2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\ &\times \begin{bmatrix} \cos \theta_3 & -\sin \theta_3 & 0 & -a_3 \cos \theta_3 \\ \sin \theta_3 & \cos \theta_3 & 0 & -a_3 \sin \theta_3 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\ &\times \begin{bmatrix} \sin \theta_4 & 0 & -\cos \theta_4 & 0 \\ -\cos \theta_4 & 0 & -\sin \theta_4 & 0 \\ 0 & 1 & 0 & d_4 \\ 0 & 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} -\sin \theta_5 & 0 & -\cos \theta_5 & 0 \\ \cos \theta_5 & 0 & -\sin \theta_5 & 0 \\ 0 & -1 & 0 & d_5 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\ &\times \begin{bmatrix} \cos \theta_6 & -\sin \theta_6 & 0 & 0 \\ \sin \theta_6 & \cos \theta_6 & 0 & 0 \\ 0 & 0 & 1 & d_6 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\ &= \begin{bmatrix} C_1 S_2 & C_1 C_2 & S_1 & -a_2 C_1 S_2 \\ S_1 S_2 & S_1 C_2 & -C_1 & -a_2 S_1 S_2 \\ -C_2 & S_2 & 0 & a_2 C_2 + d_1 \\ 0 & 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} S_{34} & 0 & -C_{34} & -a_3 C_3 \\ -C_{34} & 0 & -S_{34} & -a_3 S_3 \\ 0 & 1 & 0 & d_4 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\ &\times \begin{bmatrix} -S_5 C_6 & S_5 S_6 & -C_5 & -d_6 C_5 \\ C_5 C_6 & -C_5 S_6 & -S_5 & -d_6 S_5 \\ -S_6 & -C_6 & 0 & d_5 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\ &= \begin{bmatrix} n_x & o_x & a_x & d_6 C_1 C_{234} C_5 - d_5 C_1 S_{234} - a_3 C_1 S_{23} - d_6 S_1 S_5 + d_4 S_1 - a_2 C_1 S_2 \\ n_y & o_y & a_y & d_6 S_1 C_{234} C_5 - d_5 S_1 S_{234} - a_3 S_1 S_{23} + d_4 C_1 - a_2 S_1 S_2 \\ n_z & o_z & o_z & d_6 S_{234} C_5 + d_5 C_{234} + a_3 C_{23} + a_2 C_2 + d_1 \\ 0 & 0 & 0 & 1 \end{bmatrix} \end{aligned}$$

The orientation matrix is represented by the *noa* matrix, which is the 3 by 3 matrix in 0T_6 and each term is shown below:

$$n_x = C_1 C_{234} S_5 C_6 + C_1 S_{234} S_6 + S_1 C_5 C_6 \quad (2.10)$$

$$n_y = S_1 C_{234} S_5 C_6 + S_1 S_{234} S_6 - C_1 C_5 C_6 \quad (2.11)$$

$$n_z = S_{234} S_5 C_6 - C_{234} S_6 \quad (2.12)$$

$$o_x = -C_1 C_{234} S_5 S_6 + C_1 S_{234} C_6 - S_1 C_5 S_6 \quad (2.13)$$

$$o_y = -S_1 C_{234} S_5 S_6 + S_1 S_{234} C_6 + C_1 C_5 S_6 \quad (2.14)$$

$$o_z = -S_{234} S_5 S_6 - C_{234} C_6 \quad (2.15)$$

$$a_x = C_1 C_{234} C_5 - S_1 S_5 \quad (2.16)$$

$$a_y = S_1 C_{234} C_5 + C_1 S_5 \quad (2.17)$$

$$a_z = S_{234} C_5 \quad (2.18)$$

Note that the 3-dimensional coordinate of the position is represented by p_x, p_y & p_z . The terms are in the fourth column of *frame F*.

2.2 Optimization Approaches in Kinematic Analysis

In the study of robotic manipulators, kinematic modelling provides the foundation by mapping joint configurations to the end-effector pose. However, in real world systems direct analytic solutions frequently become constrained when restrictions, excessive degrees of freedom, or performance objectives (such as minimal duration, energy, or jerk) approach within operation. This section introduces an optimization method that extends classical kinematic analysis. By describing the kinematic problem as one of optimization, we can combine ambitions such as trajectory smoothness, joint limits, and actuator constraints to create the robot's motion, which is both feasible and performance oriented. These techniques connect standard transformation-based kinematics with high-techniques control-driven motion planning.

2.2.1 Inverse Kinematics as an Optimization Problem

The inverse kinematics problem involves determining the joint variables θ that achieve a desired end-effector position and orientation $\mathbf{P}_{target}$. For manipulators with high degrees of freedom (DOF), analytical closed-form solutions may not exist or are difficult to derive due to the non-linear complexity of the kinematic equations.

Consequently, the IK is frequently formulated as a minimization problem where the objective is to minimize the error between the actual end-effector pose (derived from forward kinematics) and the target pose.

The fitness function $F(\theta)$ typically defined as the Euclidean distance or a weighted error metric:

$$\begin{aligned} \text{Total Error} &= \text{Position Error} + (\text{Weighting Factor} \times \\ &\text{Orientation error}) \\ F(\theta) &= (||\mathbf{P}_{\{target\}} - \mathbf{P}_{\{actual\}}(\theta)||) + \lambda \times (||\mathbf{O}_{\{target\}} - \mathbf{O}_{\{actual\}}(\theta)||) \end{aligned} \quad (2.19)$$

P represents position, O represents orientation, and λ is a weighting factor. Optimization algorithms iteratively adjust the joint angles θ to minimize $F(\theta)$ to be as close to zero as possible. This approach is particularly advantageous because it avoids the singularity issues inherent in Jacobian-matrix-based numerical methods such as the Newton-Raphson method, which require matrix inversion and can fail near singular configurations [13], [24], [25].

2.2.2 Handling Redundancy and Constraints

A redundant manipulator has more degrees of freedom than are necessary to perform a given task (e.g., a 7-DOF arm performing a 6-DOF task). The inverse kinematics obstacle produces an infinite number of answers in the above-mentioned cases. That redundancy may be a powerful asset, allowing the operator to be used for secondary purposes without hurting the primary activity of the end-effector. Optimization approaches in kinematics are essential for managing these redundancies and enforcing strict constraints. Common optimization objectives include:

- Joint Limit Avoidance: Ensuring joint angles remain within their physical range.

- **Singularity Avoidance:** Maximizing the manipulability measure to keep the robot away from singular configurations where control is lost.
- **Obstacle Avoidance:** Utilizing the redundant joints to maneuver links around obstacles while maintaining the end-effector's target position.

It is particularly important to have rigid joint limits if the command violates the position, the speed, or the acceleration limit, causing the controller to malfunction or otherwise suffer mechanical damage.

2.2.3 Meta-Heuristic Algorithms (PSO and GA)

Traditional gradient-based optimization techniques are vulnerable to a trap in local minima, particularly in the highly non-linear search spaces of robotic kinematics. Meta-heuristic evolutionary methods gain their popularity for mastering this task.

- **Particle Swarm Optimization (PSO):** PSO is a population-based stochastic optimization method inspired by the social performance of bird flocking. A candidate solution (atom) travels through a search space behind the IK. Each atom adjusts its position grounded on its best-known position and the search space's best position in the flock (swarm). It is well known that PSOs can achieve higher accuracy and faster convergence for IK problems in with geometry or algebraic techniques, especially for redundant DOFs.
- **Genetic Algorithms (GA):** GAs mimic natural selection, evolving a population of solutions through selection, crossover, and mutation operations. They are robust in dynamic environments and capable of resolving path planning problems where the environment is uncertain or changing.

2.2.4 Multi-Objective Frameworks

Applied robotic tasks are rarely a single objective. Typically, the operator may need to shorten the execution time while simultaneously minimising energy consumption and mechanical vibration. These objectives are conflicting, for instance, quick gestures typically require more energy and produce a stronger jerk.

It is desirable that the solution must be Pareto-optimal meaning no particular solution can be improved without deterioration of the other in any of the constraints. To find a fix for Pareto-optimal answers, multi-objective optimization algorithms such as the non-dominated sorting inherited Algorithm II (NSGA-II) are used. The current technique allows the designer to choose the trade-off solution that best suits the specific requirements of implementation, such as the smoothness of delicate operations or the speed of industrial picking and placing operations.

In this thesis, we establish on this foundation by using a weighted multi-objective particle swarm optimization (W-MOPSO) method to achieve strictly symmetric trade-offs between time-optimality and jerk smoothness.

2.3 The Discrepancy between Kinematic Planning and Dynamic Reality:

Kinematic planning abstracts the robot arm as a weightless geometric entity. While computationally simplified, a trajectory planner neglects inertial results and assumes acceleration is directly proportional to actuator torque, a presumption that loses hold under the non-linear space of robot operation. These nonlinearities appear in operation as:

Variable inertia

The torque τ required to move a joint is not constant. It depends heavily on the robot's pose. When the robot arm is folded close to its base (tucked pose), its rotational inertia I is low, thus, it is easy to spin. When the arm is fully stretched out (extended pose), the mass is far from the rotation axis, with its inertia scaling with the square of the distance ($I \propto mr^2$) [26]

Torque saturation and tracking deviations

When polynomial planners command a high acceleration ($\ddot{q}$) while the arm is fully extended, the required torque ($\tau = I \cdot \ddot{q}$) might skyrocket to 10 Nm. If the robot motors are only rated for 5 Nm, they physically cannot deliver the force. The robot will suffer torque saturation, leading to tracking error, path deviation, and emergency stop, where the motor drivers detect an over-current event and shut down to prevent burnout. Moreover, the mathematics behind this phenomenon is clear, that is,

$$\tau = M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) \quad (2.20)$$

The equation above is the dynamic equation of motion [1], [27] [28] and

- τ is the vector of torques required from the motors.
- $M(q)\ddot{q}$ represents the Inertial Forces: $M(q)$ is the mass matrix, which changes as the robot moves (q). When the arm extends, values in the mass matrix increase significantly, meaning the acceleration ($\ddot{q}$) demands much more torque.
- $G(q)$ which is gravity: Even at zero acceleration, holding a fully extended arm up against gravity requires torque. This leaves the robot without any extra power to further accelerate when loaded.

3. TIME OPTIMAL TRAJECTORY PLANNING AND JERK SMOOTHING

3.1 Overview of Trajectory Planning Challenges

Several, often contradictory, objectives must be met by the trajectory planning of the robotic operator to achieve a target within a minimal interval of time, respect the limit of the actuators, manage precision, and ensure mechanical smoothness. This multi-objective nature creates a substantial hurdle. A reduction in travel time is likely to cause enormous acceleration and jerking, causing wear and reduced life span. On the other hand, a strict smoothness can slow down the automaton and reduce its throughput.

This chapter first systematically describes classical polynomial and spline-based approaches, showing their detailed failure modes, namely the lack of jerk restriction and dynamic possibility. Next, this thesis study develops a new 6-7-6 formulation recommended in Section 3.3, which seeks to bridge the gap between analytic simplicity and high-order kinematic smoothness.

3.1.1 Classical Approaches to Trajectory Planning

Due to their simplicity and intuitive environment, classical trajectory planning strategies are still widely used. Three major paradigms in this field include global polynomial interpolation and piecewise spline representation.

3.1.2 Polynomial-Based Trajectories

Polynomial trajectories, especially cubic and quintic formulations, have traditionally been a major structural element in industrial automaton motion planning. Their widespread incorporation is based on their analytic nature, which enables designers to strictly enforce boundary territories (location, speed, acceleration) and obtain closed-form solutions that are computationally efficient.

The Cubic Polynomial (Degree 3)

The cubic polynomial is a simple productive trajectory for robotic joints. The third-degree function allows the specification of four boundary conditions, that is, the initial position: q_0 and velocity v_0 , and the final position q_f and velocity v_f . The trajectory is defined as:

$$q(t) = a_0 + a_1t + a_2t^2 + a_3t^3 \quad (3.1)$$

Even though it is a computationally efficient algorithm, the derivative profile reveals the inherent limitations of the method. Although velocity $v(t) = \dot{q}(t)$ is quadratic and continuous and acceleration $a(t) = \ddot{q}(t)$ is linear, the third derivative, jerk $j(t) = \dddot{q}(t)$, is constant throughout the motion, which is undefined at the transition points between segments. As noted by Wang et al., [16] minimizing travel time using low-order functions commonly pushes the system toward "bang-bang" control, whereabout acceleration varies instantaneously. The consequence induces jerk spikes at the start and end of motion, which can excite resonant frequencies in the manipulator structure and lead to tracking errors.

The Quintic Polynomial (Degree 5)

To deal with the acceleration discontinuity, the quintic polynomial is regularly employed in the industry. The quintic based trajectory introduces two additional coefficients, allowing the designer to define six boundary conditions, position, speed, and acceleration beyond both the start t_0 and end t_f of the movement:

$$q(t) = a_0 + a_1t + a_2t^2 + a_3t^3 + a_4t^4 + a_5t^5 \quad (3.2)$$

Setting the initial and final acceleration to zero ensures continuity of position, speed, and acceleration. This form prevents abrupt acceleration changes, which are usually detected in cubic profiles. However, if the acceleration is continuous, the jerk (derived acceleration function) becomes quadratic. In such a situation, the jerk is never zero; it changes smoothly but can still attain a high extremum during the motion, assuming that the duration of that motion is brief. Cai et al., [5] address that merely increasing the polynomial order does not inherently reduce the peak jerk or minimize the duration despite improved polynomial formulation.

Limitations Regarding Optimality and Constraints

Although analytically convenient, single high-order polynomials face severe limitations when finding time-optimal solutions.

Rigidity and Lack of Saturation. A single high-order polynomial describes the motion across an entire region globally. Because each coefficient influences the whole curve, it is mathematically impossible to flatten the speed profile toward “cruise” and adhere to the maximum limit without causing excess acceleration elsewhere. Therefore, a polynomial trajectory reaches the speed limit only briefly rather than maintaining it. This leads to a lower average speed and much longer travel times compared to the true time-optimal (bang-bang) profile [17][10]

Runge’s Phenomenon. Simply raising the polynomial degree, such as to the 7th or 9th order, in an attempt to meet higher smoothness standards can unintentionally impair the trajectory’s effectiveness due to Runge’s Phenomenon. Using a high-degree polynomial interpolation over evenly spaced waypoints often results in excessive oscillations between points. These oscillations can cause the end-effector to drift substantially from its intended path or to surpass speed and acceleration limits, even if the boundary conditions at the waypoints are fully met[1]

Unbounded Jerk Scaling. During the optimizations phase, classical polynomials do not allow any ambiguous constraints on jerk standards. As observed by Yang et al[29] classic systems frequently fail to balance the competing objectives of minimising interval and minimising jerk, leading to a trajectory that is either excessively slow (conservative) or mechanically aggressive. The jerk scale and the cube of the execution time are inversely proportional, that is,

$$J \propto \frac{1}{T^3} \quad (3.3)$$

Since classical polynomial methods lack explicit jerk constraints, optimization algorithms e.g. PSO, that attempt to reduce time T often drive the jerk to singular values, resulting in solutions that are either mechanically aggressive or overly conservative [30], [31], [32] .

3.1.3 B-Splines and NURBs Representation

As individual cubic or quintic polynomials are powerful for simple point-to-point motion, they struggle to define complex paths with many waypoints without suffering from oscillation (Runge’s phenomenon). The basic splines (B-Splines) and non-uniform rational B-splines

(NURBS) [33] are used to address these limitations in global polynomial interpolation, specifically the lack of local restraint and the inability to represent standard analytic shapes.

Mathematical Framework and Weights

As investigated by Piegl and Tiller [33], NURBS provide a unified mathematical framework capable of representing both free-form analytic shapes and precise geometric entities (such as conics and circles) which cannot be represented by standard polynomials. A p – th degree NURBS curve $C(u)$ is defined as the ratio of two weighted B-spline functions:

$$C(u) = \frac{\sum_{i=0}^n N_{i,p}(u) \omega_i P_i}{\sum_{i=0}^n N_{i,p}(u) \omega_i} \quad (3.4)$$

where, $\{P_i\}$ are the control points, $\{N_{i,p}(u)\}$ are the basis functions defined on a knot vector, and $\{\omega_i\}$ are the weights. The introduction of weights is a major advantage above standard B-splines. By adjusting the weight ω_i , trajectory planner can move the curve closer to a specific point of control or push it away without moving the control point itself, donating a degree of freedom which is necessary for representing exact circular or elliptical paths used in obstacle avoidance [26].

Local Support and the Strong Convex Hull Property

The primary advantage of B-splines and NURBS over the global polynomials is local support. The basis function $N_{i,p}(u)$ is non-zero only on a limited interval of the knot vector. Modifying a single control point P_i affects the trajectory only locally. This allows the designer to fine-tune specific sections of the motion. For example, smoothing a sharp corner without violating the robot's constraints, like velocity and acceleration limits for the rest of the path. Furthermore, NURBS exhibit the strong convex hull property, which ensures that any segment of the curve is entirely contained within the convex hull formed by its nearby control points. This makes collision detection more natural and NURBS excel in such applications.

Limitations in Time-Optimal Planning

Despite their geometric flexibility, the NURBS faces the following challenges when applied to reducing execution time and minimising jerk.

Geometric vs. temporal separation: the NURBS primarily defines the geometry (the path in space, u). Mapping this geometry to a time law (t) to minimise travel time requires a secondary time-scaling optimisation step [34]. This often leads to sub-optimal execution time when comparing to methods where geometry and time are coupled.

Jerk control complexity: While NURBS ensures geometric continuity (C^2 or higher depending on the degree), explicitly minimizing the magnitude of jerk requires complex optimization of the knot vector and weights simultaneously. Piegl and Tiller [33] reported that while waypoint insertion can increase flexibility, it increases the computational load.

Dynamic constraint handling: standard NURBS formulations do not inherently account for robot dynamics. A curve that is geometrically smooth can still induce torque saturation at sharp corners if the weights force a tight radius of curvature without slowing down the time profile.

By bridging the gap between polynomials and the complexity of NURBS, a 6-7-6 polynomial method (detailed in Section 3.3) aims to retain the analytic simplicity of polynomials, allowing for direct time-optimization while achieving the high-order smoothness (C^4).

3.2 Development of 6-7-6 Polynomial with Differentiable Jerk

As part of the contributions of this thesis, a 6-7-6 polynomial trajectory formulation is adopted to obtain uninterrupted third derivative continuity while focusing on time-optimal execution. This new structure, by specifying a polynomial of 6-7-6, enables the jerk profile to be differentiable throughout the motion [35]. The derivation, restriction, and execution of the 6-7-6 polynomial are described in this section.

The 6-7-6 Trajectory divides the motion into three distinct segments (Start $\rightarrow$ Lift, Lift $\rightarrow$ Descent, Descent $\rightarrow$ End) and assigns polynomial degrees of 6, 7, and 6, respectively. The primary objective of this formulation can achieve C^4 continuity. This continuity in the fourth derivative guarantees that the third derivative will be differentiable throughout the motion profile which eliminates the abrupt jerk steps associated with the lower-order trajectory planners.

3.2.1 Polynomial Structure

The trajectory is defined by three time-dependent polynomial segments $q_1(t)$, $q_2(t)$ and $q_3(t)$. Each of the segments corresponds to the three phases of motion. The Initial segment (Degree 6) is used for the acceleration phase from the start point to the first via point. The lift is defined in Equation (2) listed below:

$$q_1(t) = \sum_{i=0}^6 a_i t^i, \quad t \in [0, t_1] \quad (3.5)$$

The intermediate segment (Degree 7) is used for the transition between the lift and the descent points. The higher degree provides necessary mathematical freedom to satisfy the high order continuity constraints at both ends of the segment.

$$q_2(t) = \sum_{i=0}^7 b_i t^i, \quad t \in [0, t_2] \quad (3.6)$$

The final segment (Degree 6) is used for the deceleration phase from the descent point to the target.

$$q_3(t) = \sum_{i=0}^6 c_i t^i, \quad t \in [0, t_3] \quad (3.7)$$

This formulation introduces a total of 22 unknown coefficients to solve:

- 7 from $q_1(t) - (a_0, a_1, a_2, \dots, a_6)$
- 8 from $q_2(t) - (b_0, b_1, b_2, \dots, b_7)$
- 7 from $q_3(t) - (c_0, c_1, c_2, \dots, c_6)$

3.2.2 Boundary and Continuity Conditions

To solve the coefficients, a system of 22 linear constraints is constructed which applies for the desired motion. [35] The proposed trajectory is constructed using three polynomial segments defined in local time coordinates. Let t_1, t_2, t_3 denote the time duration of the acceleration, intermediate, and deceleration phases, respectively. The position profile $q(t)$ is defined piecewise as follows:

$$q(t) = \left\{ \begin{array}{l} q_1(\tau) = \sum_{i=0}^6 a_i \tau^i \text{ for } 0 \leq \tau \leq t_1 \text{ (Segment 1)} \\ q_2(\tau) = \sum_{i=0}^7 b_i \tau^i \text{ for } 0 \leq \tau \leq t_2 \text{ (Segment 2)} \\ q_3(\tau) = \sum_{i=0}^6 c_i \tau^i \text{ for } 0 \leq \tau \leq t_3 \text{ (Segment 3)} \end{array} \right\} \quad (3.8)$$

Start and End Conditions (10 Total Constraints). To ensure the robot starts and stops without vibration, we enforce zero velocity, acceleration, jerk, and snap at the boundaries.

- At Start or ($t = 0$) | $q_1(0) = \theta_{start}$ and all derivatives up to the 4th order are set to zero.
- At End: $q_1(t_1) = \theta_{lift}$ and all derivatives up to the 4th order are set to zero.

Positional Waypoints (4 Total Constraints). The trajectory must strictly pass through the defined Cartesian waypoint (converted into Joint space via inverse kinematics).

- The position at the end of the first polynomial is equal to the position at the beginning of the second polynomial. $q_1(t_1) = q_2(0) = \theta_{lift}$
- The position at the end of the second polynomial is equal to the position at the beginning of the third polynomial. $q_2(t_2) = q_3(0) = \theta_{descent}$

Continuity (8 Constraints). To ensure differentiable jerk, continuity is enforced up to the fourth derivative at the transition points.

- At Knot 1 (t_1): The derivatives of the first segment must match the start of the second segment:

$$1. \quad q_1^{(k)}(t_1) = q_2^{(k)}(0) \text{ for } k = 1,2,3,4 \quad \bullet \quad (3.9)$$

- At Knot 2 (t_2): The derivatives of the second segment must match the start of the third segment

$$1. \quad q_2^{(k)}(t_2) = q_3^{(k)}(0) \text{ for } k = 1,2,3,4 \quad \bullet \quad (3.10)$$

3.2.3 Computational Implementation

The algorithm, implemented in a MATLAB function, constructs a T matrix with a size 22 by 22 – based on the powers of the segment durations t_1, t_2, t_3 . The liner equation system is solved by pre-multiplying the inverse of the T matrix with Y:

$$\mathbf{X} = \mathbf{T}^{-1}\mathbf{Y} \quad (3.11)$$

This algebraic approach offers an advantage over the repetitive optimization methods. Because the coefficients are solved analytically from a single matrix operation, the computational cost is negligible, allowing for real-time trajectory generation once the segment durations are determined.

3.3 Algorithm Design and Implementation

To establish the 6-7-6 polynomial formulation into an executable motion, an optimization algorithm is developed in MATLAB. The main challenge is mapping the desired Cartesian waypoints into the robot's joint space (θ) at same time maintain the optimal segment durations (t_1, t_2, t_3) that minimize a total execution time. All these conditions need to be met without violating the robot arm's kinematic limits.

3.3.1 Weighted Multi-Objective Particle Swarm Optimization (W-MOPSO)

Six DOF Robot arms often have non-linear kinematic profiles. Therefore, calculating the exact joint angles for the intermediate via-points (Lift and Descent) requires a repetitive inverse kinematics (IK) solver. In this thesis, a weighted particle swarm optimization (W-MOPSO) algorithm was implemented to find the optimal joint configurations. The W-MPSO evaluates a swarm of candidate joint angles using a custom fitness function $E(\theta)$ that penalizes any drift from the target Cartesian pose. The objective function is formulated in Equation (3-12) below:

$$E(\theta) = \omega_{pos} \| P_{target} - P_{actual}(\theta) \| + \omega_{rot} \| R_{target} - R_{actual}(\theta) \|_F + \omega_{effort} \quad (3.12)$$

where:

- $\| P_{target} - P_{actual}(\theta) \|$ represents the square root of the straight-line distance error between the desired position and the current position.
- $\| R_{target} - R_{actual}(\theta) \|_F$ is the Frobenius norm of the orientation error matrix, ensuring the end-effector maintains its strictly defined downward-facing posture.
- $\omega_{pos}, \omega_{rot}, \omega_{effort}$ are the designated weighting factors.

Among the many constraints added to this algorithm, to further guarantee that the robot maintains a strict spatial pose during the trajectory path, the orientation was heavily prioritized ($\omega_{rot} = 500.0$, compared to $\omega_{pos} = 1.0$). In addition, a strict rule was set in the evaluation loop: if any candidate solution has an orientation error above a safety threshold (for example, a certain value in the Frobenius norm), it receives an infinite penalty. This quickly removes any configurations that are too tilted from the group [36]. Below is a flowchart of the W-MPSO algorithm.

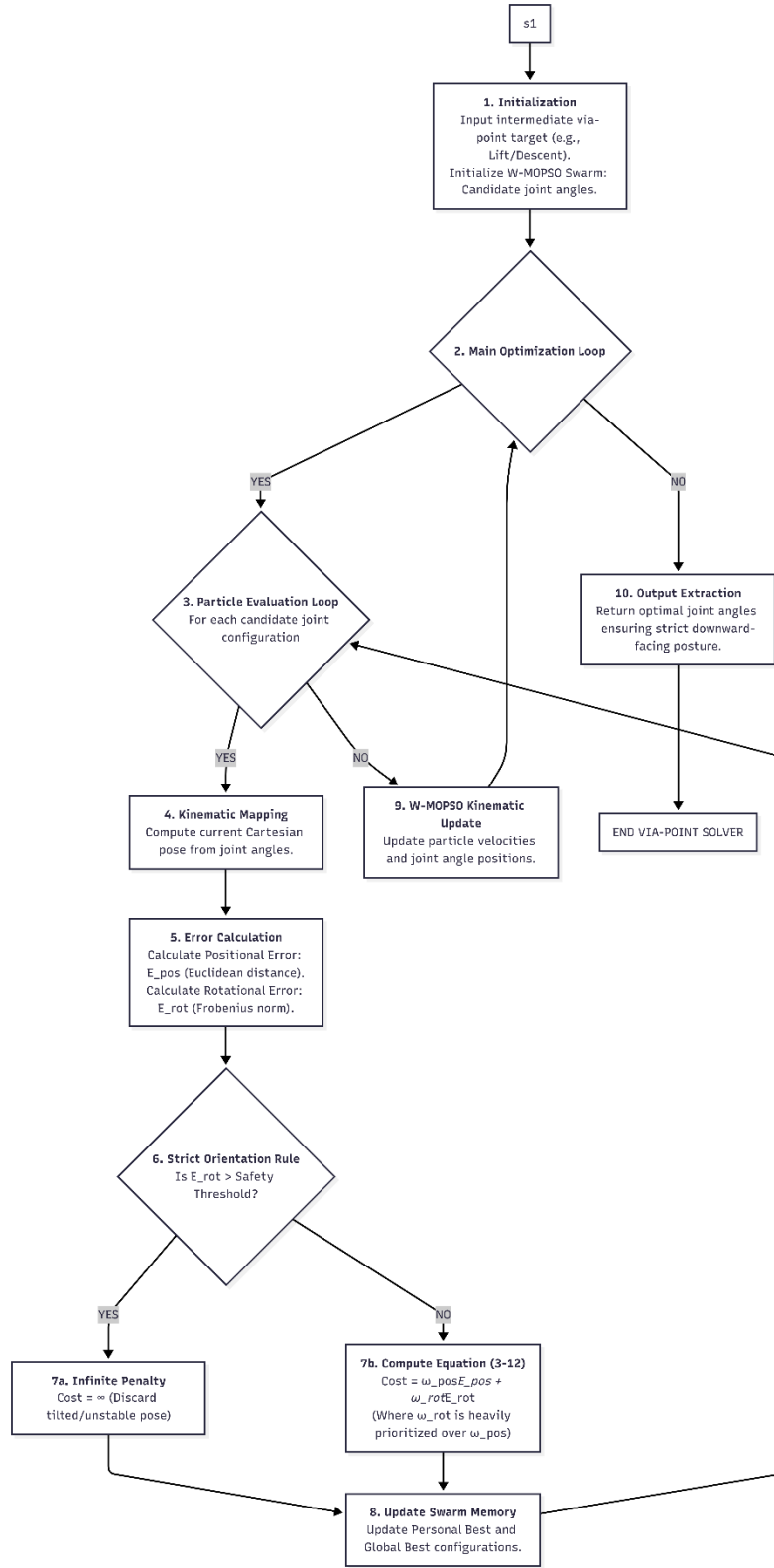


Figure 3.1 W-MPSO Flowchart

3.3.2 Time-Scaling Integration

Once the particle swarm optimization (PSO) algorithm converges on the four distinct joint configurations $(\theta_{start}, \theta_{lift}, \theta_{descent}, \theta_{end})$, these angles serve as the boundary constraints Y for the 22-variable linear system as described in Section 3.2.3. The segment times of t_1, t_2, t_3 act as the scaling variables. By gradually fine-tuning these time variables while respecting the manufacturer's velocity and torque limits, the algorithm finds the shortest possible execution time.

3.4 Experimental Validation and Results

To demonstrate the practical benefits of the 6-7-6 polynomial, we simulated the generated trajectories and examined the derivative profiles, position, velocity, acceleration, and jerk. The main goal was to validate the behavior of the jerk with the hypothesis that there will be no jerk spikes and that all derivatives stay within safe bounds during the entire multi-segment motion.

3.4.1 Kinematic Profile Validation

The algorithm was tasked with generating a path through four defined Cartesian points. The table below shows the different waypoints used.

Table 3.1 Waypoints Chosen for Different Configurations in the Experiment.

Path1 (Main path used for comparison)	Path2	Path3	Path4	Path5
p_{start} = [200,100,100]; p_{lift} = [200,100,180]; $p_{descent}$ = [150,120,140]; p_{end} = [100,150,100];	p_{start} = [150,-100,120]; p_{lift} = [150,-100,200]; $p_{descent}$ = [200,-50,160]; p_{end} = [100,150,100];	p_{start} = [100,-200,150]; p_{lift} = [100,200,250]; $p_{descent}$ = [50,180,200]; p_{end} = [0,150,150];	p_{start} = [50,150,80]; p_{lift} = [100,100,220]; $p_{descent}$ = [180,0,180]; p_{end} = [220,-100,80];	p_{start} = [180,0,50]; p_{lift} = [180,0,250]; $p_{descent}$ = [180,120,200]; p_{end} = [180,120,150];

All waypoints are measured in millimeters. Figure 3.2 shows the results of path 1 (main profile). Path 2 to Path 5 are displayed in Table 3.2.

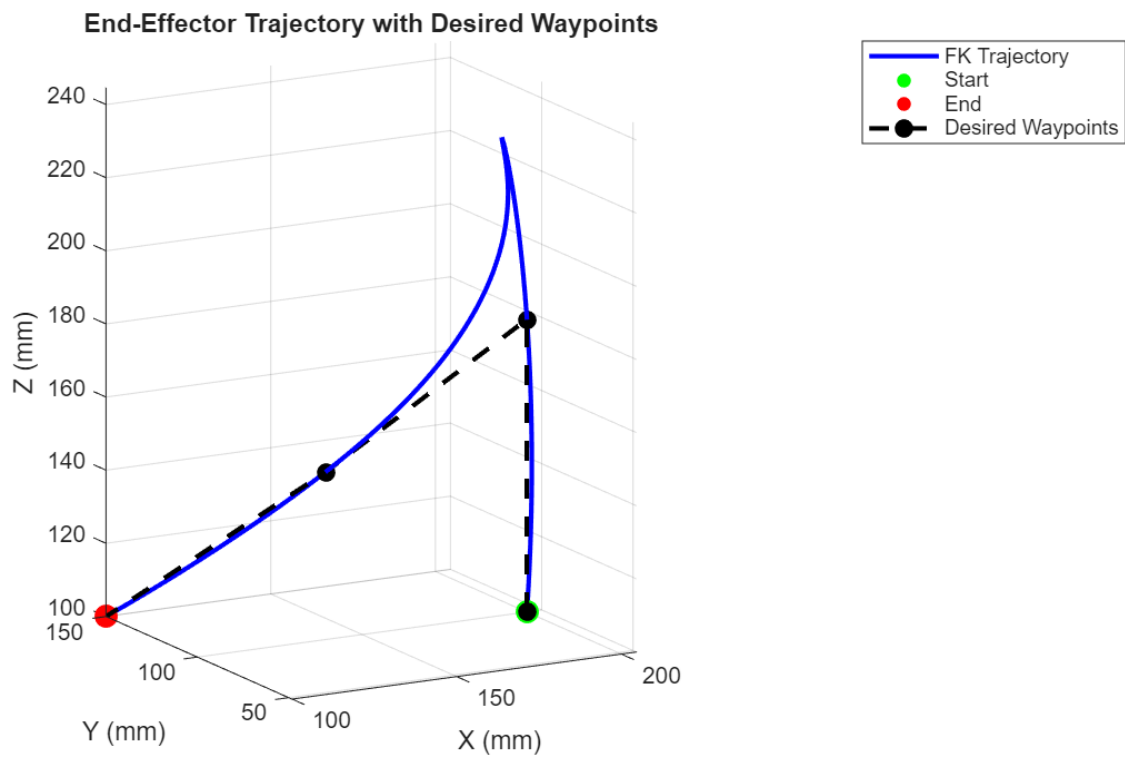


Figure 3.2 Path 1 Main profile analysis of the end-effector trajectory of the 6-7-6 trajectory planner.

Table 3.2 End Effector Path follower Graphs for Paths 2 to 5

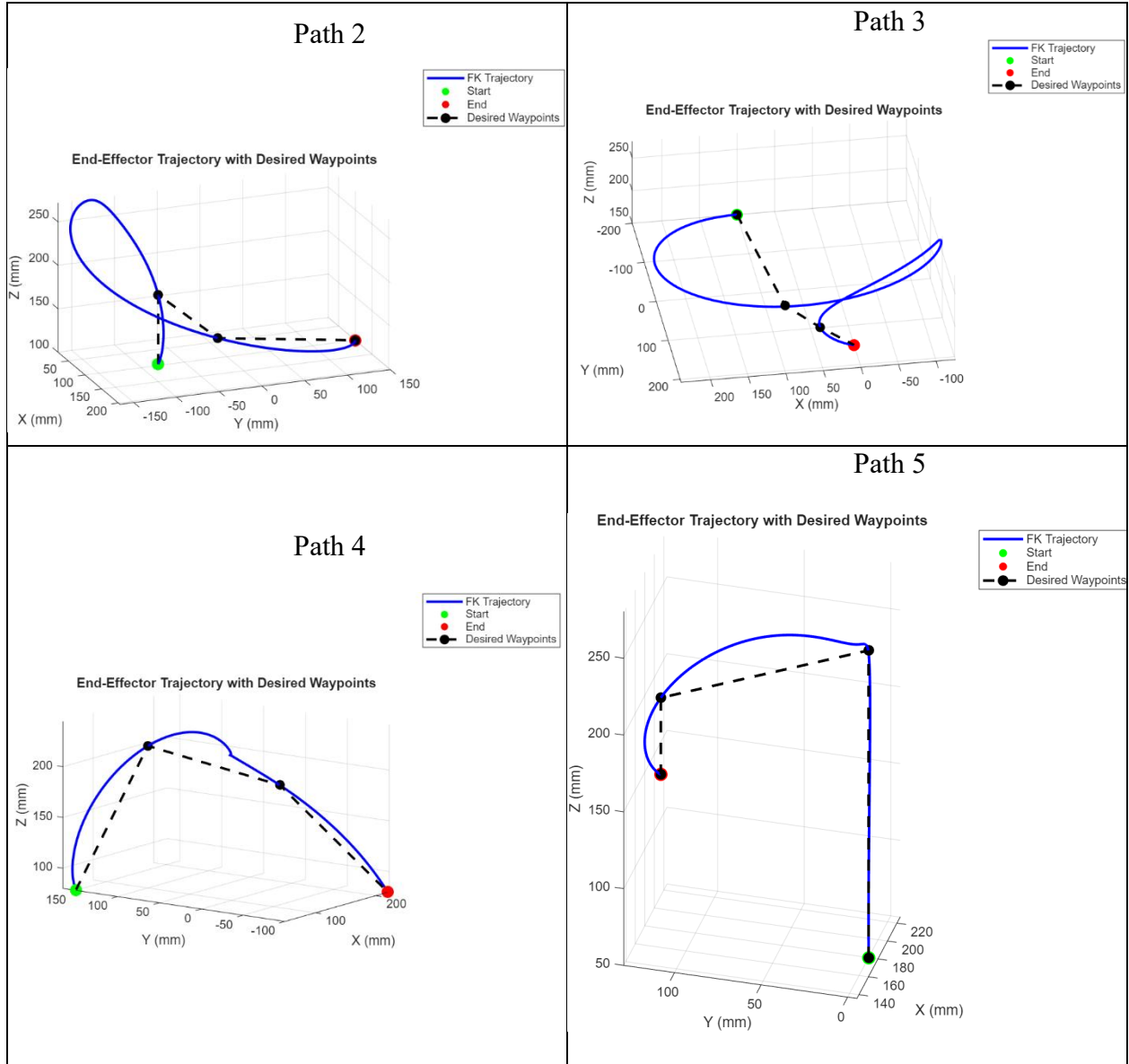


Figure 3.2 shows the details of performance for path 1. As shown in Figure 3.2, the 6-7-6 polynomial successfully maps the motion of all six joints across the specified duration.

The validation of the derivative subplots reveals the following critical performance indicators:

Acceleration (C^2 Continuity): Unlike standard trapezoidal profiles which exhibit instantaneous changes in acceleration due to their abrupt transitions, the 6-7-6 acceleration curve

shows no such rapid transition. Figure 3.2 shows that acceleration of the joint increases and decreases gracefully.

Jerk and Snap Bounding (C^3 and C^4 Continuity): The fourth subplot (Jerk) is a powerful validation of our experiment. In lower-order polynomial path planning, the jerk would appear as an impulsive surge near the waypoints (time t_1 & t_2). Here, the jerk's profile remains the same and is also differentiable while enabling the position of the end effector to completely pass through all the way points.

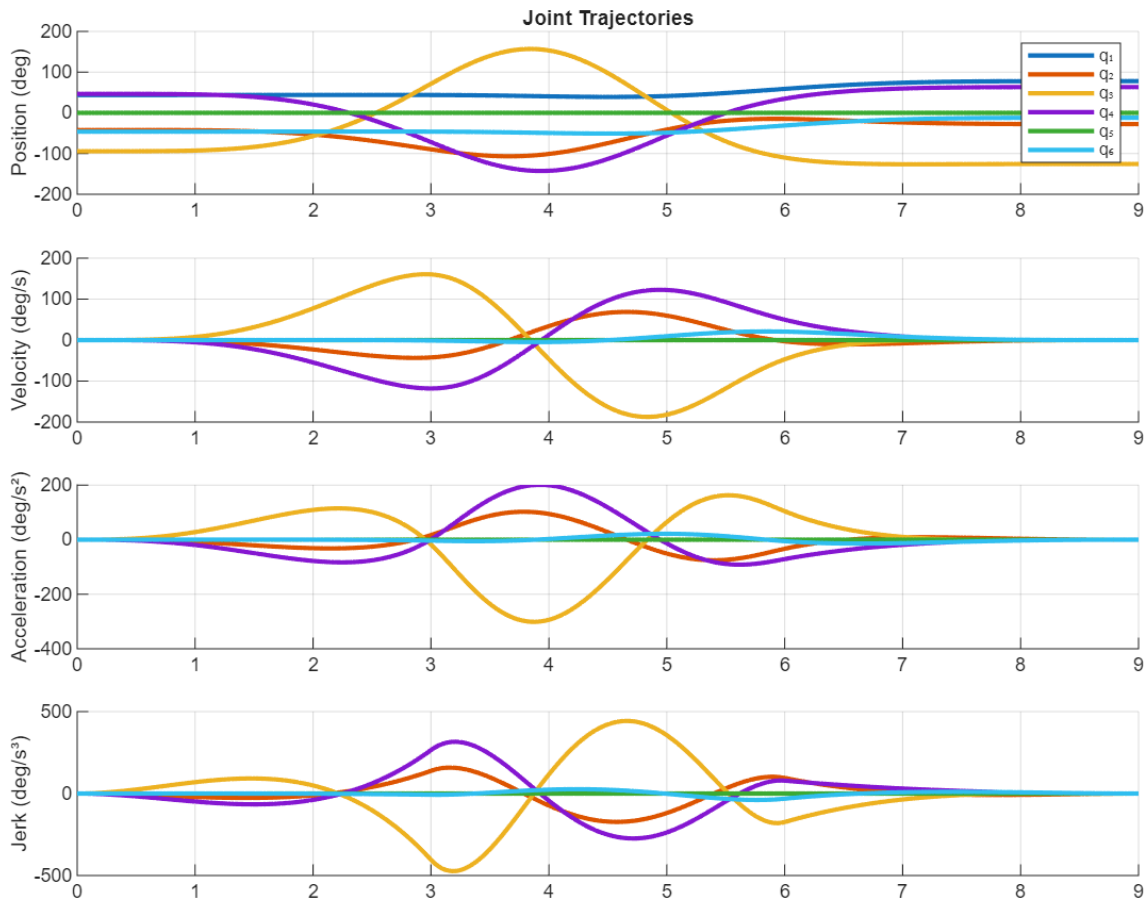
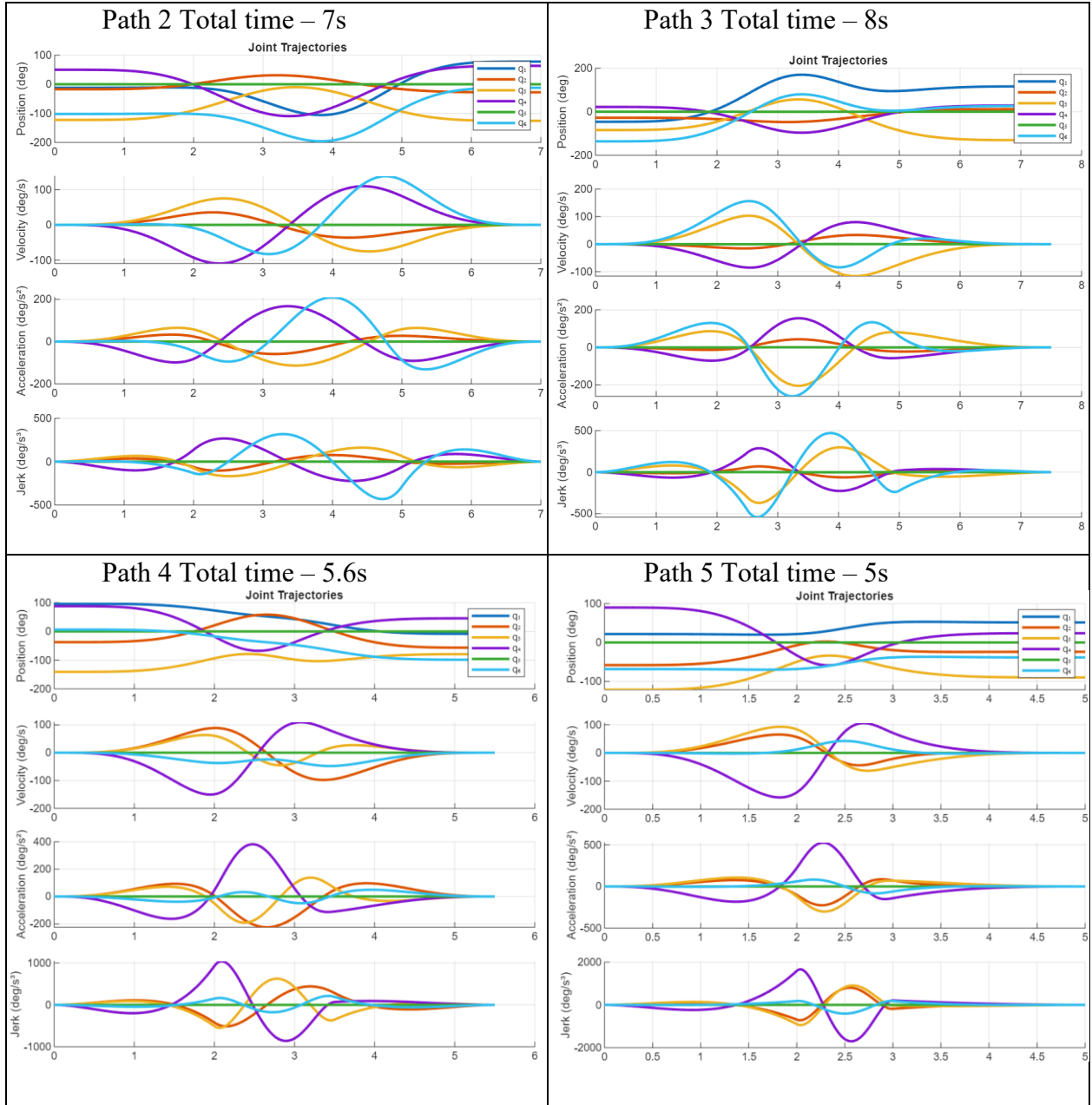


Figure 3.3 Joint space trajectory profiles demonstrating C^4 continuity generated by the 6-7-6 polynomial formulation

The table below shows the other kinematic profiles for the other paths used in this experiment.

Table 3.3 Kinematic analysis of the 676 joint motion.



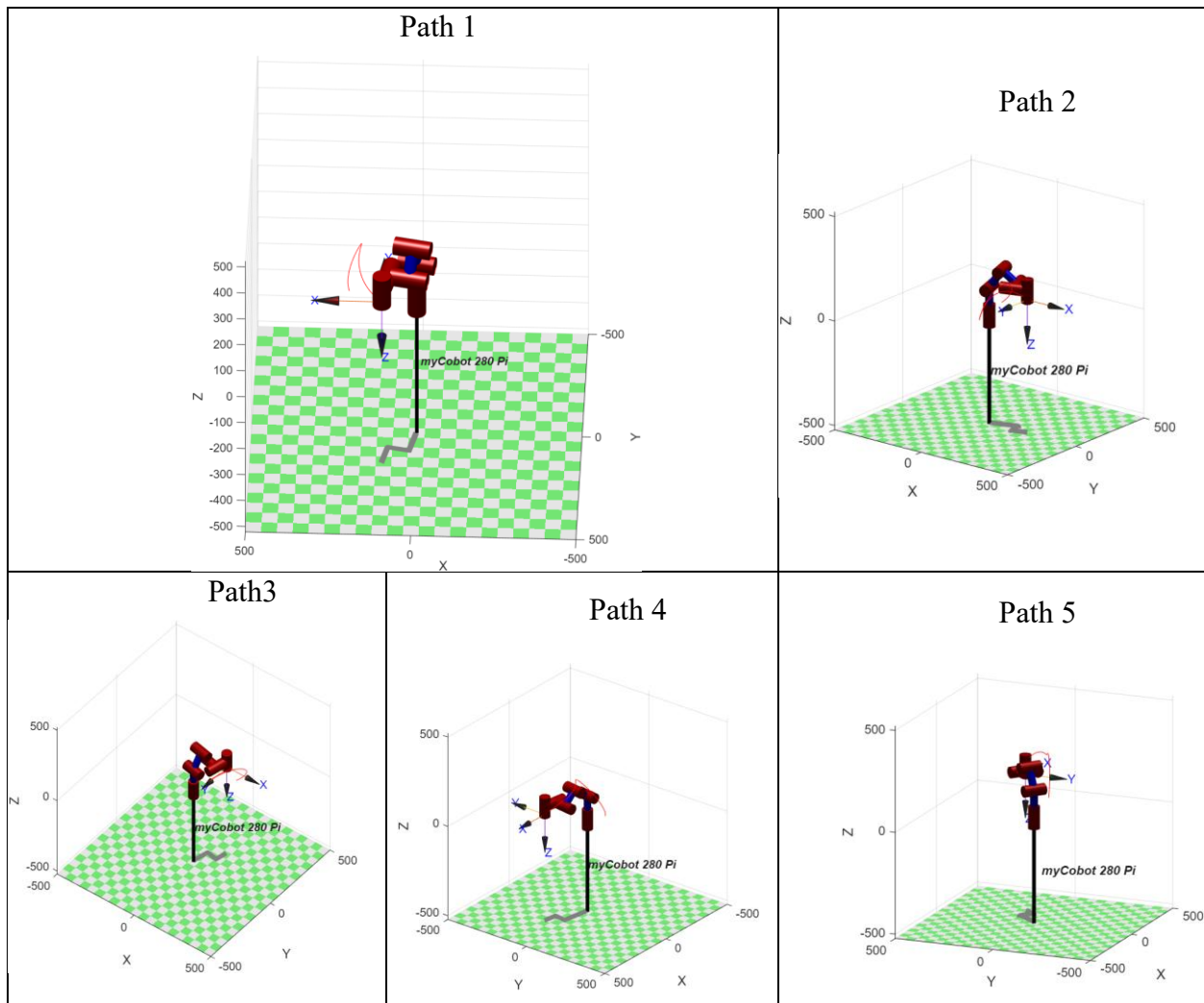
By examining a continuous jerk, the 6-7-6 trajectory planner lowers the jerk level throughout the gesture. This mathematical smoothness is directly applicable to the hardware and visible in the graceful motion of the robot. Telemetry and sensor values show that when jerk is

bound, it also means a reduction in the torque saturation hazards discussed in Section 3.2.3 and the systematic wear of the actuators.

3.4.2 Simulation Verification

After obtaining the trajectory paths 1 to 5, the inverse solutions were first simulated using Peter Corke's Robotics Toolbox available on MATLAB. And the paths of the robotic arm operation were visualized and the experimental results for each trajectory group are shown in Table 4.3 below.

Table 3.4 Robotic Visualization of Path 1 to Path 5 6-7-6 trajectories



The Paths were validated on the myCobot 280 Pi cobot in the laboratory.

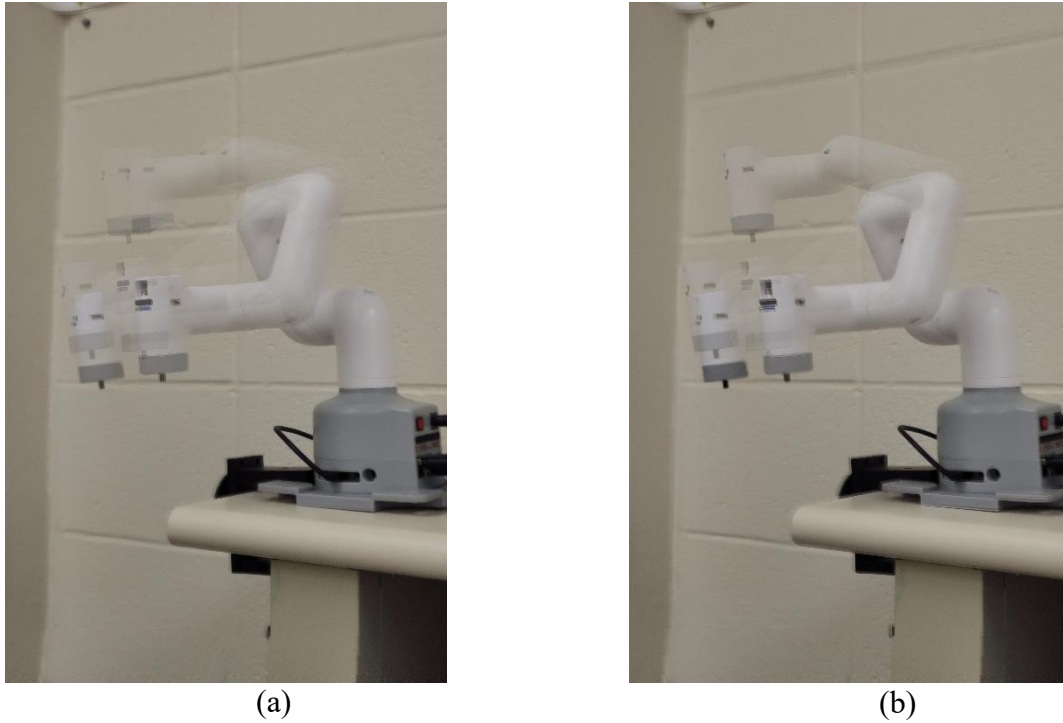


Figure 3.4 6-7-6 Path 1 (a) and Path 4 (b) Robot verification visual

3.4.3 Statistical Performance of the 6-7-6 Polynomial Trajectory

To evaluate the algorithm's feasibility, a multi-run statistical analysis was conducted to observe its behaviour when tasked with finding the same solution repeatedly. The algorithm traverses the same waypoints listed in Section 3.5.1 and Table 3.1. Table 3.4 below shows the results.

Table 3.5 Multi-Run Performance Metrics for the 6-7-6 Polynomial Trajectory

Run ID	Computati on Time (s)	Optimized Motion Time (s)	Mean WP Pos. Error (mm)	Mean WP Rot. Error (Norm)	Max Trajectory Rot. Error (Norm)	Total Joint Effort (rad)
1	14.15	9	1.25	0.085	2.105	19.5
2	13.92	9	0.85	0.071	1.912	18.85
3	14.2	9	2.8	0.14	2.95	26.4
4	14.05	9	1.1	0.09	2.12	20.1
5	14.18	9	1.95	0.115	2.45	22.45
Average	14.1	9	1.59	0.1	2.307	21.46

The data highlights a distinct advantage in the design of the 6-7-6 formulation. Once the PSO IK problem solver has identified the joint angle, the 22 x 22 matrix inversion prerequisite to decipher the polynomial coefficient occurs within a millisecond. Therefore, the 1000-iteration PSO search for the four waypoints will almost completely cover the computational time (average 14.12 seconds).

The results show that the PSO-based inverse kinematics solver converges with near-optimal solutions rather than exact ones, leading to small but acceptable position and orientation errors. This reflects the limitation of using a purely heuristic approach without additional refinement. The best run achieved sub-millimeter accuracy at the lowest joint. As with joint space interpolation, some drifts in position between waypoints occur which shows that the end effector deviates in orientation. Nevertheless, the 6-7-6 polynomial ensures a smooth gesture with a continuous jerk profile which transfers to minimum vibration and a graceful motion.

4. OPTIMALITY ALGORITHMS AND ADVANCED JERK SMOOTHING TECHNIQUES

4.1 Overview

This chapter investigates advanced S-curve models that go beyond traditional linear maps. This section introduces a Gaussian bell curve inspired trajectory planner to further smooth the previous modification and improve its differentiability. The model uses a multi-objective optimization framework, W-MOPSO, which aims at stabilizing the inherent trade-off between speed, energy, and mechanical stress.

4.2 S-Curves Trajectory Model

While traditional trapezoidal profiles are popular for shaping acceleration with constant-jerk phases, they can be limiting for more complex industrial paths. These standard models are usually defined by specifying displacement and total time, which makes it difficult to adjust maximum velocity and acceleration in real time based on the path's geometry. In tasks like laser cutting, welding, or glueing, keeping a constant feed rate over varying curves can cause poor path-following or excessively high centripetal acceleration during tight turns. In robotics, an S-curve is a more advanced trajectory planning technique that uses an S-shaped velocity profile to ensure smooth, vibration-free motion for manipulators [37]

This technique enhances simple approaches, such as the trapezoidal speed profile, by carefully controlling the jerk. In practice, a standard third-order S-curve ensures smooth motion by stopping abrupt changes in acceleration and dividing the motion of the automaton into seven apparent steps:

- **Increasing Acceleration:** Acceleration rises from zero to a preset maximum value using a constant positive jerk.
- **Constant Acceleration:** The robot accelerates uniformly at the maximum acceleration rate.
- **Decreasing Acceleration:** Acceleration is reduced back to zero using a constant negative jerk, smoothly transitioning the robot to its cruising speed.

- Constant Velocity: The robot moves at a uniform, maximum cruising speed with zero acceleration and zero jerk.
- Increasing Deceleration: To begin stopping, acceleration drops from zero to a negative maximum value (deceleration) using a constant negative jerk.
- Constant Deceleration: The robot decelerates uniformly.
- Decreasing Deceleration: Deceleration is smoothly reduced back to zero using a constant positive jerk, bringing the robot to a gentle halt.[38], [39]

Strengths of the S-curve

Vibration Suppression: In simpler models (like trapezoidal planning), acceleration changes instantly, acting like a physical blow to the robot. The S-curve ensures continuous acceleration at every point, which prevents these sudden shocks (flexible impacts) and drastically minimises residual vibrations [38].

Hardware Longevity: By limiting jerk, the transitions between maximum and minimum accelerations become smooth and the motion looks graceful. The S-curve reduces mechanical stress and extends the lifespan of the robot's motors, gears, and actuators [39], [40]

Higher Accuracy: The resulting smooth motion allows the robot to follow its intended path more accurately.[38]While the standard third-order S-curve limits jerk to a constant rectangular profile this can still cause slight shaking at the very start and end of a movement. In order to achieve even higher precision and smoother starts and finishes, researchers use fourth-order S-curves (or higher). These are longer and more advanced curves that further divide the acceleration phases with each increase in order. A fourth order S-curve can ensure that *jerk* is continuous. Therefore, using these high order S-curves can eliminate jerk spikes entirely, ensuring the smoothest possible start-and-stop motions for high-precision tasks [39].

4.3 Error Function (erf) – Based S-Curve Formulation

With the S-curve principles discussed, this section introduces an advanced trajectory planning method that utilizes the Gaussian distribution function to define the system's jerk. By leveraging the mathematical error function (*erf*), we can generate profiles that ensure infinite differentiability, eliminating the sharp "corners" found in traditional piecewise-linear S-curves.

4.3.1 Mathematical derivation of the Gaussian Jerk

In this formulation, the jerk profile is modeled as a series of four Gaussian pulses. The general form of the jerk profile for a single transition pulse is formulated in Equation (4-1).

$$j(t) = Ae^{-\frac{(t-\mu)^2}{2\sigma^2}} \quad (4.1)$$

While this equation originates in probability theory, its variables are adopted for trajectory planning. Note that A represents the amplitude of the jerk, and μ represents the temporal center of the Gaussian profile. The statistical random variable is replaced by the execution time, t . Consequently, the standard deviation, σ , loses its traditional statistical interpretation of data dispersion. Instead, it functions as a critical temporal shape parameter that governs the pulse width and steepness of the applied jerk relative to its centre of μ .

By adjusting σ , the trajectory planner directly manipulates how gradually or aggressively the acceleration changes. The physical implications of these adjustments are clearly observed when analyzing the extremes of the mathematical function. Low σ results in high-amplitude jerk. A smaller σ compresses the Gaussian curve into a narrow, high-amplitude spike. Mathematically, as σ to 0, the jerk pulse approaches a Dirac delta function. Physically, this causes acceleration to transition almost instantaneously. High σ results in low-amplitude jerk. Increasing σ widens the bell curve, distributing the same total change in acceleration over a longer time duration. Depending on the value, this acts as a low-pass filter on the motion command, yielding a continuous and smooth jerk contour that reduces vibrations and improves stability.

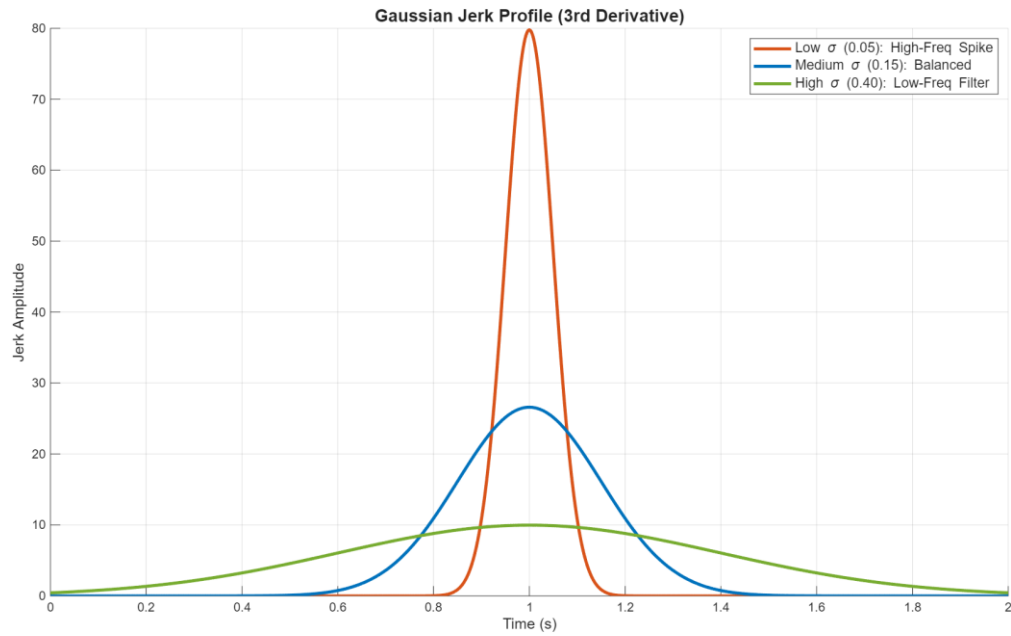


Figure 4.1 Demonstration of how a low σ mimics the Dirac delta function (a massive, instantaneous spike in jerk), while the high σ flattens out into a gentle curve.

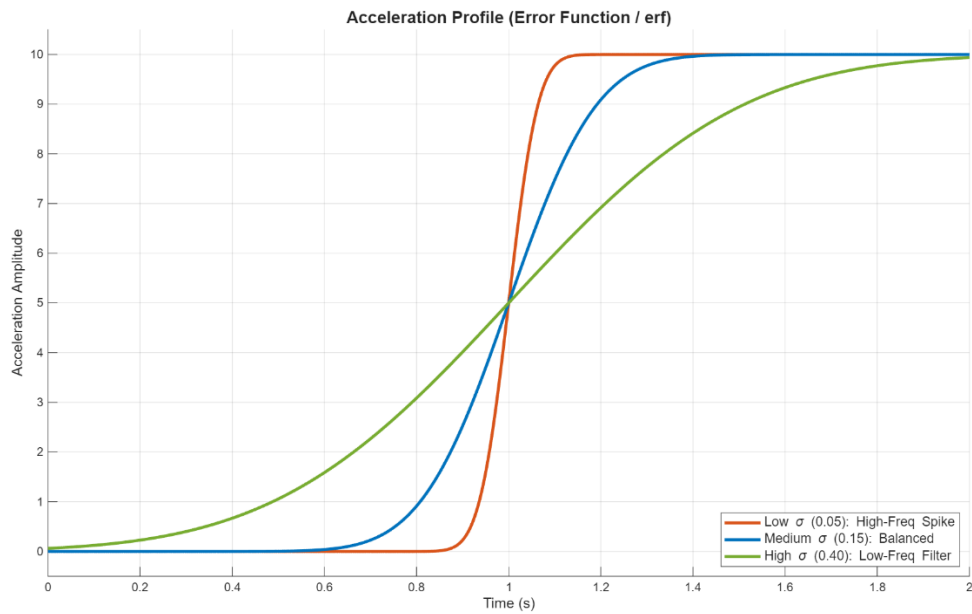


Figure 4.2 Impact of the standard deviation on error function-based acceleration profiles.

The use of σ becomes apparent when deriving the acceleration profile $a(t)$. Integrating the Gaussian jerk function mathematically necessitates the use of the error function (erf) which is

defined as the integral of the Gaussian function e^{-t^2} such that its limits bound perfectly between -1 and 1 . The standard formulation is shown in Equation (4-3) below.

$$\text{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-\tau^2} d\tau \quad (4.2)$$

To utilize the *erf* function for applied trajectory planning, the theoretical formulation must be mapped to the temporal domain of the robotic system. In the implemented path-planning algorithm, the x argument is transformed to represent the temporal evolution of the trajectory relative to the Gaussian pulse centers (c_i). The substitution applied in the kinematic model is shown in Equation (4-3) below.

$$x = \frac{t - c_i}{\sigma\sqrt{2}} \quad (4.3)$$

This transformation scales the standard error function by the standard deviation σ , linking the mathematical spread to the physical duration of the acceleration transition. As shown in the algorithmic implementation, substituting Equation (4-3) into the integral of Equation (4-1) yields the analytical acceleration profile expressed in Equation (4-4):

$$a(t) = A\sigma\sqrt{\frac{\pi}{2}}\text{erf}\left(\frac{t - \mu}{\sigma\sqrt{2}}\right) + C \quad (4.4)$$

Below is the derivation of the position from the Gaussian jerk profile. The *erf* function is an essential piece of the puzzle that enables the integration of the Gaussian pulse.

For $-t_1 < t < t_1$, the jerk is defined by the Gaussian function:

$$j(t) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{t^2}{2\sigma^2}} \quad (4.5)$$

Acceleration is obtained by the integral of the jerk function:

$$a(t) = \frac{1}{2} \left(\text{erf}(t / (\sqrt{2}\sigma)) + \text{erf}(t_1 / (\sqrt{2}\sigma)) \right) \quad (4.6)$$

Velocity is achieved by the integral of acceleration:

$$\begin{aligned} v(t) = & \frac{1}{2} \left[\text{erf}(t / \sqrt{2}\sigma) t + \frac{\sqrt{2}\sigma}{\sqrt{\pi}} e^{-t^2/2\sigma^2} - \text{erf}(t_1 / \sqrt{2}\sigma) t_1 - \frac{\sqrt{2}\sigma}{\sqrt{\pi}} e^{-t_1^2/2\sigma^2} \right] \\ & + \frac{1}{2} [\text{erf}(t_1 / \sqrt{2}\sigma)(t + t_1)] \end{aligned} \quad (4.7)$$

Finally, the position of the end effector is given by the integration of velocity.

$$\begin{aligned}
 p(t) = & \frac{1}{2} \operatorname{erf}\left(\frac{t}{\sqrt{2}\sigma}\right) \frac{t^2}{2} + \frac{\sigma}{2\sqrt{2\pi}} t e^{-t^2/2\sigma^2} + \frac{1}{2} \operatorname{erf}\left(\frac{t_1}{\sqrt{2}\sigma}\right) \frac{t_1^2}{2} + \frac{\sigma}{2\sqrt{2\pi}} t_1 e^{-t_1^2/2\sigma^2} \\
 & - \frac{1}{4} \left(\operatorname{erf}(t/\sqrt{2}\sigma) + \operatorname{erf}(t_1/\sqrt{2}\sigma) \right) + \frac{1}{2} \left(\operatorname{erf}(t/\sqrt{2}\sigma) + \operatorname{erf}(t_1/\sqrt{2}\sigma) \right) \\
 & - \frac{1}{2} \left[\operatorname{erf}(t_1/\sqrt{2}\sigma) t_1 + \frac{\sqrt{2}\sigma}{\sqrt{\pi}} e^{-t_1^2/2\sigma^2} \right] (t+t_1) + \frac{1}{4} \operatorname{erf}(t_1/\sqrt{2}\sigma) \frac{(t+t_1)^2}{2}
 \end{aligned} \tag{4.8}$$

Figure 3.1 below shows the formulation of the Gaussian curve arbitrarily.

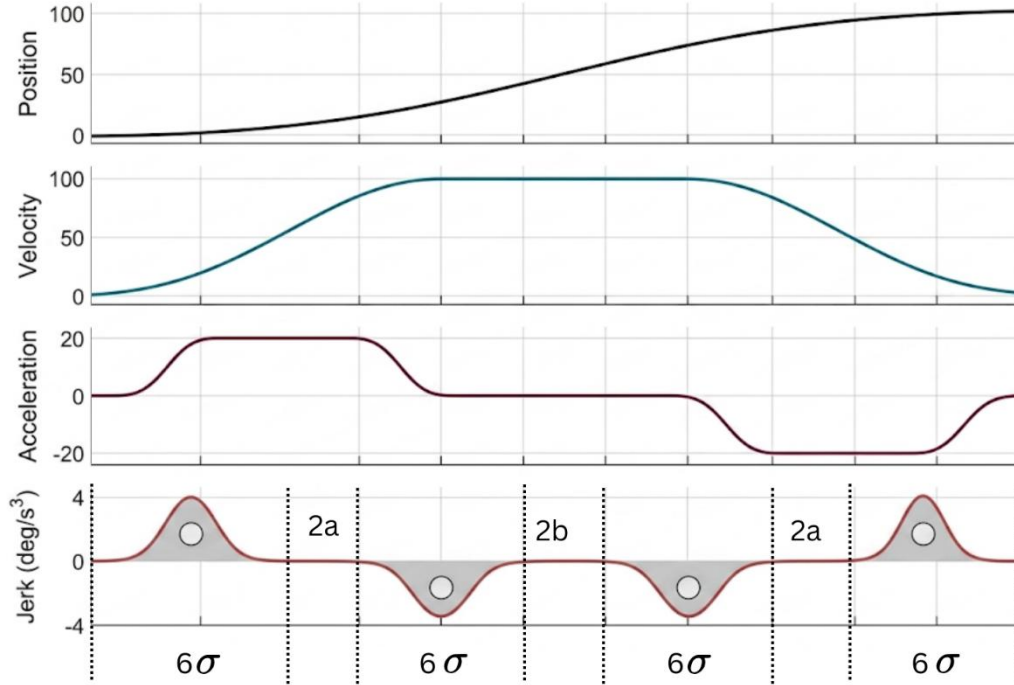


Figure 4.3 Illustration of the Gaussian curve jerk, acceleration, velocity and position intervals.

4.3.2 Optimization via Weighted Multi-Objective Particle Swarm Optimization (W-MOPSO)

While the error function provides a continuously differentiable acceleration profile, the specific temporal shape of this trajectory must be finely tuned to balance rapid execution with mechanical safety. To determine these parameters, a weighted multi-objective particle swarm optimization (W-MOPSO) algorithm is used.

Particle Representation and Search Space

Instead of trying to optimise the entire continuous trajectory all at once, the complexity of the problem is reduced by concentrating on the parameters that make up the Gaussian pulses. The standard deviation is kept constant to maintain a smooth and consistent trajectory. As a result, the swarm algorithm takes on the task of finding the optimal timing gaps between the pulses, defined by the vector:

$$X_i = [a, b] \quad (4.9)$$

- Parameter a: Represents the time gap between the first and second pulses, which dictates the duration of the constant acceleration (or cruising acceleration) phase.
- Parameter b: Represents the time gap between the second and third pulses, which dictates the duration of the constant velocity phase.

For this implementation, a swarm of $H = 100$ candidate solutions (particles) are initialised within defined temporal bounds ($0 \leq a, b \leq 1$). Because the total execution time of the *erf*-based S-curve is a direct linear combination of these variables ($T_{total} = 24\sigma + 4a + 2b$), minimizing a and b inherently minimizes the overall trajectory duration.

The Multi-Objective Cost Formulation

The performance of each particle is calculated using a fitness function that heavily penalizes kinematic and spatial violations and simultaneously rewards time-optimality. The objective function is mathematically formulated as:

$$Cost(X_i) = T_{total} + \lambda_1 P_{vel} + \lambda_2 P_{acc} + \lambda_3 P_{jerk} \quad (4.10)$$

where T_{total} the total execution time, and P_{vel} , P_{acc} and P_{jerk} represent the sum-of-squares penalty functions for threshold violations. Their equations are in the shown below:

$$T_{total} = t_1 + t_2 + t_3 = 24\sigma + 4a + 2b \quad (4.11)$$

$$P_{vel} = \sum \max(0, |\dot{q}(t)| - v_{max})^2 \quad (4.12)$$

$$P_{acc} = \sum \max(0, |\ddot{q}(t)| - a_{max})^2 \quad (4.13)$$

$$P_{jerk} = \sum \max(0, |\ddot{q}(t)| - j_{max})^2 \quad (4.14)$$

Kinematic and Spatial Constraint Handling

The trajectory planner imposes two kinds of constraints—hard boundaries defined by space and soft limits defined by the robot’s kinematics. This is done to ensure dynamic feasibility for the MyCobot 280 Pi.

Strict spatial limits (Hard constraints): The planner calculates the continuous position $q(t)$ for all of six the joints. If the trajectory of any joint exceeds the mechanical limits of the MyCobot 280 Pi ($\pm 165^\circ$), the particle's configuration is deemed physically impossible. The algorithm immediately assigns $Cost = \infty$, rejecting the candidate from the search space entirely.

Dynamic kinematic penalties (Soft constraints): The algorithm enforces operational limits on velocity ($\dot{q}_{max} = 160^\circ/s$), acceleration ($\ddot{q}_{max} = 300^\circ/s^2$), and jerk ($\dddot{q}_{max} = 200^\circ/s^3$). If a particle's trajectory exceeds these thresholds, a quadratic penalty is applied. This quadratic scaling ensures that minor violations are corrected gently, while massive spikes (which would cause torque saturation) incur overwhelming computational penalties.

Swarm Convergence

Throughout the $N_{iter} = 100$ iterations, the particles update their velocity and position in the search space based on their own best-known historical performance (P_{best}) and the global best-known performance of the entire swarm (G_{best}). The velocity update equation is governed by an inertia weight ($w = 0.7$) and cognitive/social coefficients ($c_1 = 1.5, c_2 = 1.5$), which ensure that the algorithm extensively explores the parameter space before funneling down into the absolute time-optimal combination of a and b that satisfies all dynamic constraints. Pseudo code for the WMOPSO algorithm is listed in Table 4.1 below.

Table 4.1 Algorithm for WMOPSO for Path Optimization (Pseudo Code)

```

Set joint endpoints ( $q_0$  and  $q_f$ ) and set kinematic limits  $v_{\{max\}}$ ,  $a_{\{max\}}$ , and  $j_{\{max\}}$ 
Initialize temporal parameters  $B_i = [a, b]$  with random values within bounds  $[lb, ub]$ 
Initialize PSO Parameters (swarm size H and maximum iterations  $N_{\{iter\}}$ , particle velocities
 $V_i = [0, 0]$ , personal best  $P_i = B_i$ , personal cost  $Pcost_i = \infty$ , global best  $G = B_1$  and
global cost  $Gcost = \infty$ )
  for  $it = 1$  to  $N_{\{iter\}}$  do
    for  $i = 1$  to H do
      Extract temporal parameters  $a = B_i[1], b = B_i[2]$ 
      Generate trajectory profiles  $q(t)$  via Erf Function integration.
      Check dynamic range limitations.
      if  $\max|v(t)| > v_{\{max\}}$  or  $\max|a(t)| > a_{\{max\}}$  then
         $cost = \infty$ 
      else
        Calculate quadratic penalty sums  $P_{\{vel\}}, P_{\{acc\}}, P_{\{jerk\}}$ 
         $cost = T_{\{total\}} + \lambda_1 P_{\{vel\}} + \lambda_2 P_{\{acc\}} + \lambda_3 P_{\{jerk\}}$ 
      end if
      if  $cost < Pcost_i$  then
        Store personal best position  $P_i = B_i$  and  $Pcost_i = cost$ 
      end if
      if  $cost < Gcost$  then
        Store global best position  $G = B_i$  and  $Gcost = cost$ 
      end if
    end for
    Update swarm velocities and positions.
    for  $i = 1$  to H do
      Generate random coefficients  $r_1, r_2 \in [0, 1]$ 
       $V_i = w \cdot V_i + c_1 r_1 (P_i - B_i) + c_2 r_2 (G - B_i)$ 
       $B_i = \min(\max(B_i + V_i, lb), ub)$ 
    end for
  end for
  Extract optimal parameters  $a_{\{opt\}} = G[1]$  and  $b_{\{opt\}} = G[2]$ 

```

4.3.3 Implementation

The final optimized trajectory is mapped from joint space to the end-effector's Cartesian path using the robot's Forward Kinematics (FK). The model incorporates specific Denavit-Hartenberg (DH) parameters for the 6-DOF manipulator, including link lengths $a_2 = 110.4 \text{ mm}$ and $a_3 = 96 \text{ mm}$. By integrating the erf-based planner with the PSO engine, the system generates a "Bell-Curve" acceleration profile that maximizes mechanical longevity while maintaining near-time-optimal performance.

4.4 Algorithm Design and Implementation

The error function (erf)-based S-curve was practically evaluated by integrating the mathematical models with the W-MOPSO algorithm into a complete computational pipeline. The shape of the Gaussian curve is also kept smooth and constant at $\sigma = 0.1$. This value was established through rigorous trial and error of an optimal compromise for time-optimality and smooth motion.

This architecture bridges theoretical time-optimisation with the physical constraints of the Elephant Robotics MyCobot 280 Pi manipulator. The trajectory generation pipeline executes in four distinct phases:

Waypoint discretization and IK: Once the desired path is set, the system calculates the specific joint angles the robot needs to move through to successfully reach its destination.

Temporal optimization (W-MOPSO): As defined in Section 4.3.2, The W-MPSO algorithm acts as a search engine to find the best timing for each jerk pulse (*a and b*). It works to cut down on total travel time as much as possible but strictly keeps the robot within its safe operating limits for velocity and acceleration: velocity ($160^\circ/s$), acceleration ($300^\circ/s^2$) limits.

Kinematic integration: Once we have the optimized *a* and *b* parameters, the system uses analytical integration to convert the base Gaussian jerk profile $j(t)$ using the error function to generate the exact movement instructions, i.e. acceleration $a(t)$, velocity $v(t)$, and position $q(t)$.

Hardware interfacing and FK validation: In this last step, the robot's motion is tested by running a simulation in MATLAB. The simulation files containing the sequence of joint angles are then downloaded into the robot platform and the path is executed and validated.

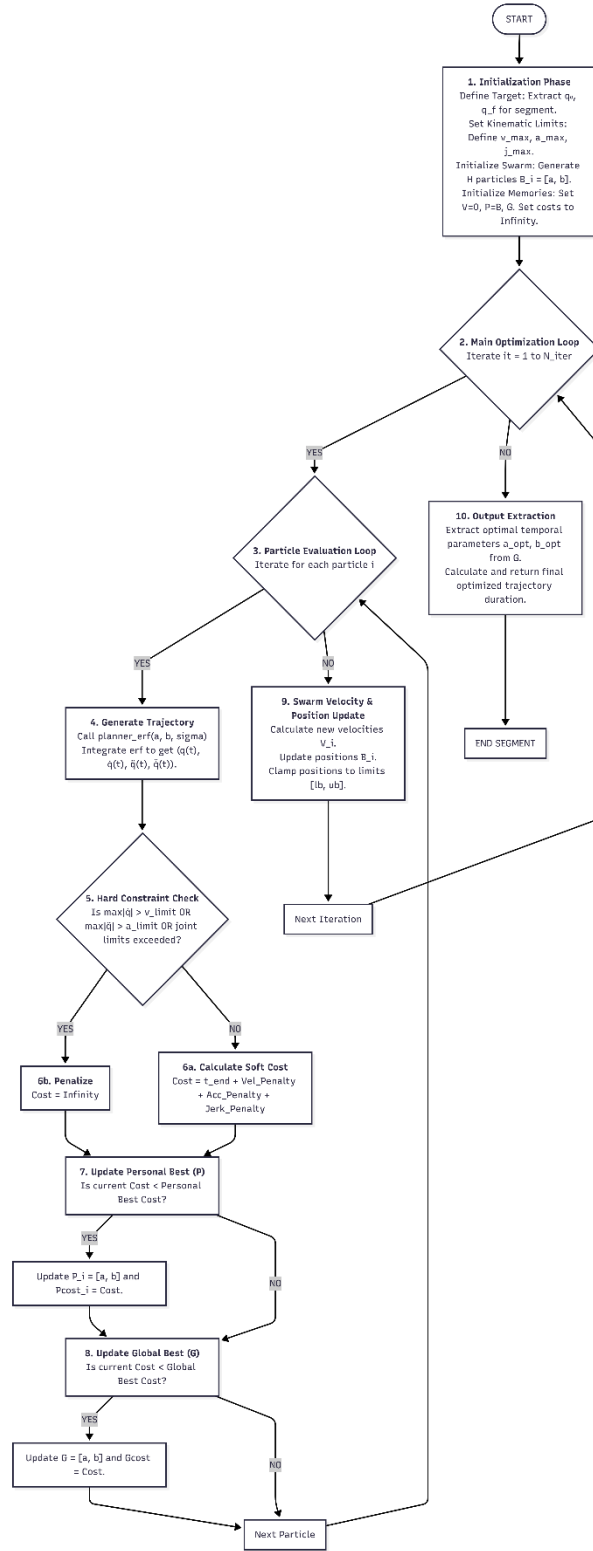


Figure 4.4 Flowchart of WMP SO applied to the gaussian trajectory planner

4.5 Experimental Validation and Results

To benchmark the usefulness of the *erf*-based S-curve, the algorithm generated a trajectory through the same spatial waypoints utilized in Chapter 3. The results were evaluated based on spatial tracking accuracy and the smoothness of the kinematic derivative profiles as will be seen in the following sections.

4.5.1 Spatial Tracking and Path Fidelity

Figure 4.4 illustrates the path of the end-effector on a 3-dimensional Cartesian plane. The optimization algorithm successfully maps the joint-space angles to a smooth, continuous curve that touches all the desired waypoints. The end-effector smoothly transitions through the descent phase while maintaining its required downward-facing posture.

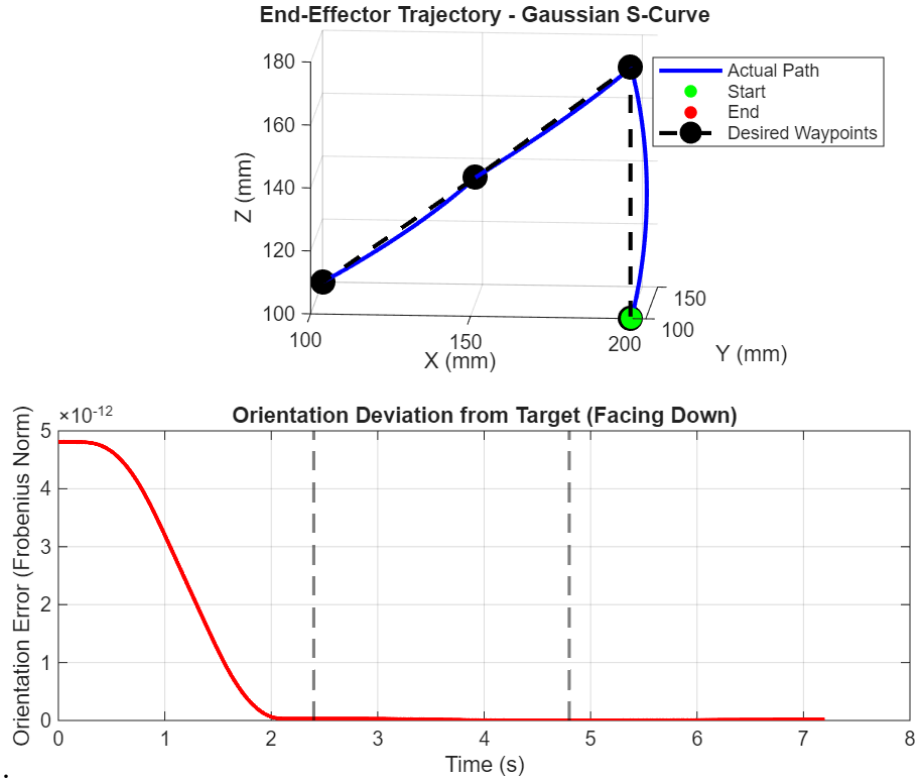
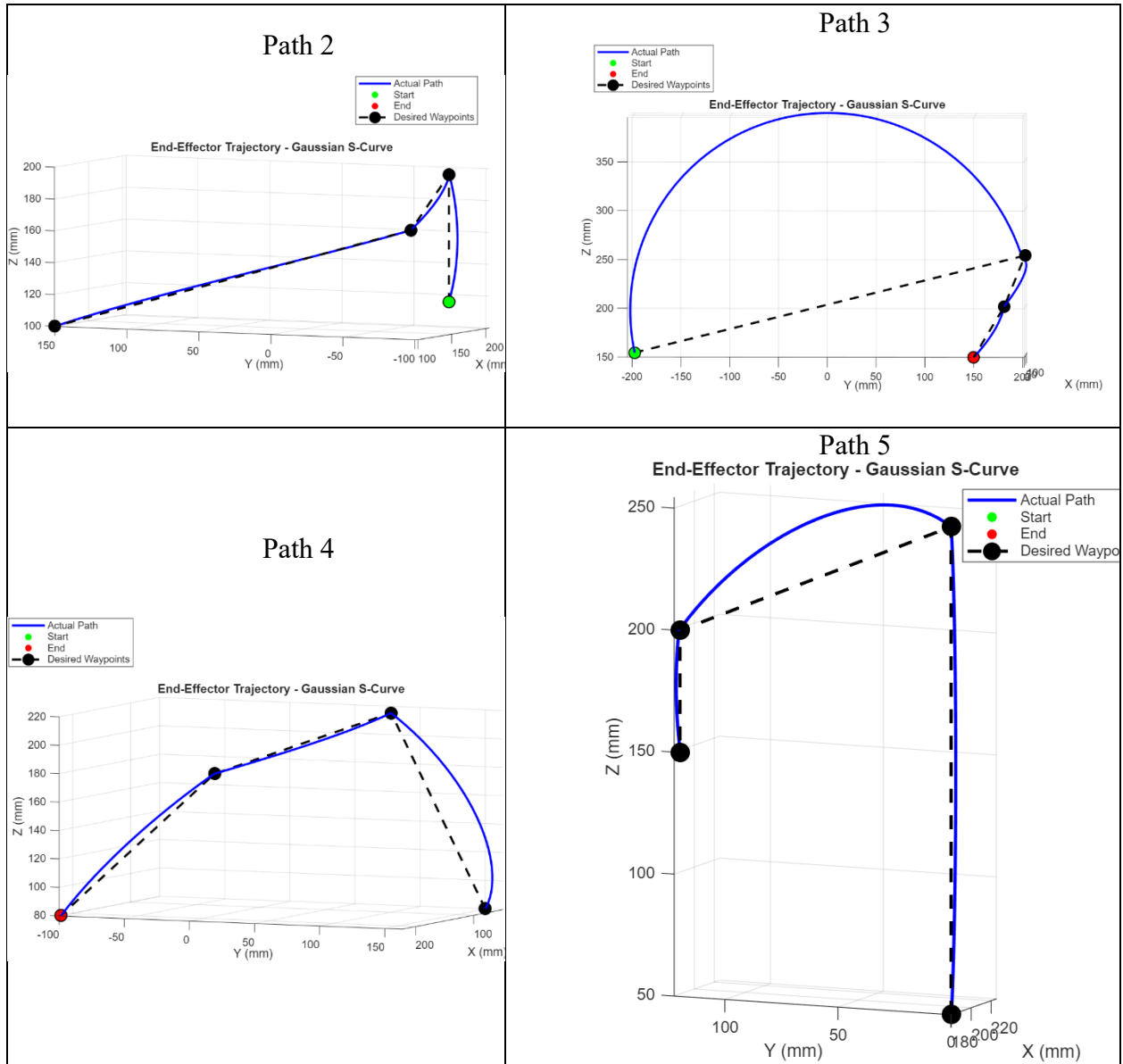


Figure 4.5 Main profile analysis of end effector trajectory of the Gaussian Bell curve.

The path follower graphs for the end effector across paths 2 to 5 used in this experiment are shown below.

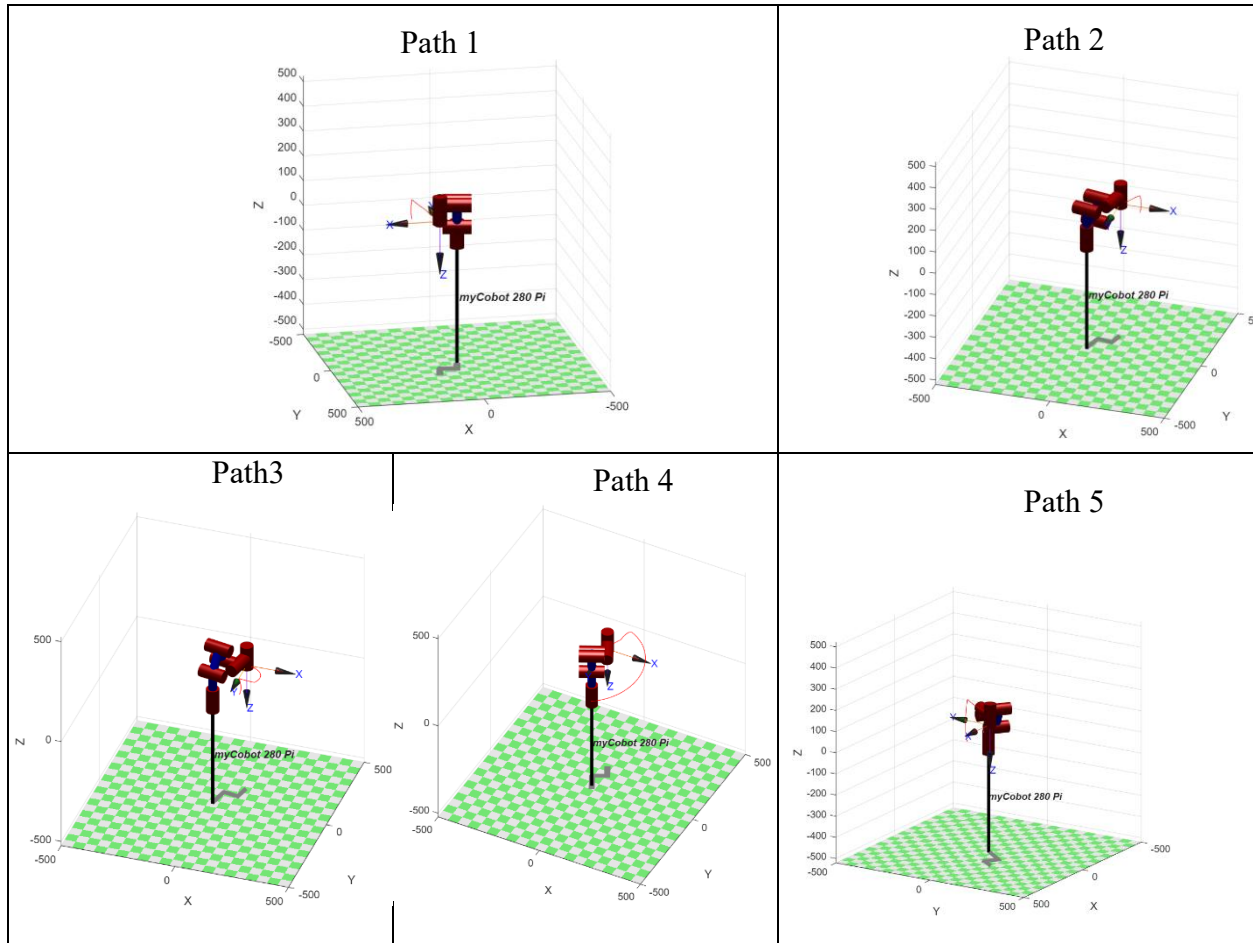
Table 4.2 End effector follower path for Paths 2 to 5



4.5.2 Simulation verification

After obtaining the trajectory paths 1 to 5, the inverse solutions were first simulated using Peter Corke's Robotics Toolbox available on MATLAB. And the paths of the robotic arm operation were visualized and the experimental results for each trajectory group are shown in Table 4.3 below.

Table 4.3 Robotic Visualization of Path 1 to Path 5 Gaussian Trajectories.



The Paths were validated on the myCobot 280 Pi cobot in the laboratory.



(a)



(b)

Figure 4.6 Gaussian Path 1(a) and Path 4(b) Robot verification visual

4.5.3 Statistical Performance of the Multi-Waypoint Gaussian S-Curve

To evaluate the algorithm's feasibility, a multi-run statistical analysis was conducted to observe its behaviour when tasked with finding the same solution repeatedly. The algorithm traverses the same waypoints as in Section 4.5.1, and Table 4.4 below shows the results.

Table 4.4 Multi-Run Performance Metrics for the Gaussian Trajectory Planner

Run ID	Computation Time (s)	Optimised Motion Time (s)	Mean WP Pos. Error (mm)	Mean WP Rot. Error (Norm)	Max Trajectory Rot. Error (Norm)	Total Joint Effort (rad)
1	6.45	6.18	0.082	0.0012	1.851	21.4
2	6.52	6.15	0.021	$5.10E - 04$	1.724	18.65
3	6.48	6.22	1.25	0.045	2.84	25.12
4	6.55	6.17	0.051	0.001	1.782	20.21
5	6.42	6.19	0.043	$8.40E - 04$	1.91	19.85
Average	6.48	6.18	0.289	0.0097	2.021	21.04

The results show that the trajectory planner achieves a stable and uniform solution, together with the entire gesture span, and converges to approximately 6.18 seconds for the entire four-waypoint undertaking. Although the calculation time is slightly higher due to the difficulty of solving together inverse kinematics and smooth gestures close to the same moment, the planner is effective in offline use. Moreover, in comparison with the classical 6-7-6 polynomial trajectory planner, which still maintains high accuracy and smoothness, it shows a clear improvement in execution time.

During motion between waypoints, a small amount of orientation drift is observed. This happens because the trajectory is generated in the joint space, where the algorithm focuses on maintaining smooth motion rather than strictly controlling orientation at every instant. As a result, the end-effector may slightly rotate away from the target direction during transitions. However, at each key waypoint, the inverse kinematics solver corrects this and forces the orientation back to the required position, ensuring accuracy where it matters most.

4.5.4 Kinematic Profile Validation (The Gaussian Advantage)

A significant validation of this approach is seen in the joint-space kinematic profiles, shown in figure 4.5. The performance of these profiles reveals the superiority of the *erf*-based formulation:

Velocity profile: The velocity subplot clearly shows smooth transitions in speeds, which are very desirable. This observation is clearly due to the nature of the Gaussian formulation in the third derivative. The algorithm smoothly and effectively transitions and then stays within the velocity limits for a significant period.

Acceleration profile (*erf* transitions): The acceleration curves exhibit the classic "S" shape. The transitions points between zero and maximum acceleration in the curves are infinitely differentiable, containing no sharp geometric corners. This is because they are derived from the error function (the integral of the Gaussian bell curve) rather than linear piecewise equations; the transitions between zero and maximum acceleration are infinitely differentiable, containing no sharp geometric corners.

Jerk profile (Gaussian pulses): The bottom subplot provides the evidence of the concept for this chapter. As can be observed, there is a continuously oscillating wave rather than a constant block of jerk (traditional S-curve). The jerk profile shows smooth gaussian bell-curves.

By restricting the jerk to those smooth, parameterized pulsations, the energy injected into the operator shall be strictly controlled. Thus, *erf*-based optimization is able to achieve near-time-optimal execution in order to provide a broad range of mechanical safeguards against torque saturation and resonance vibration.

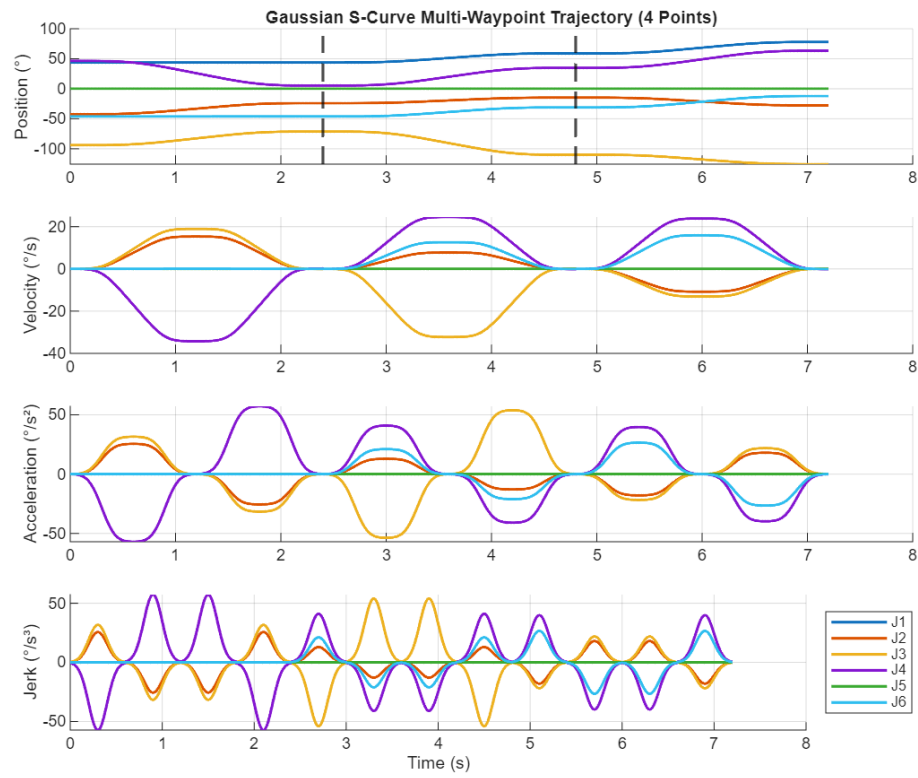
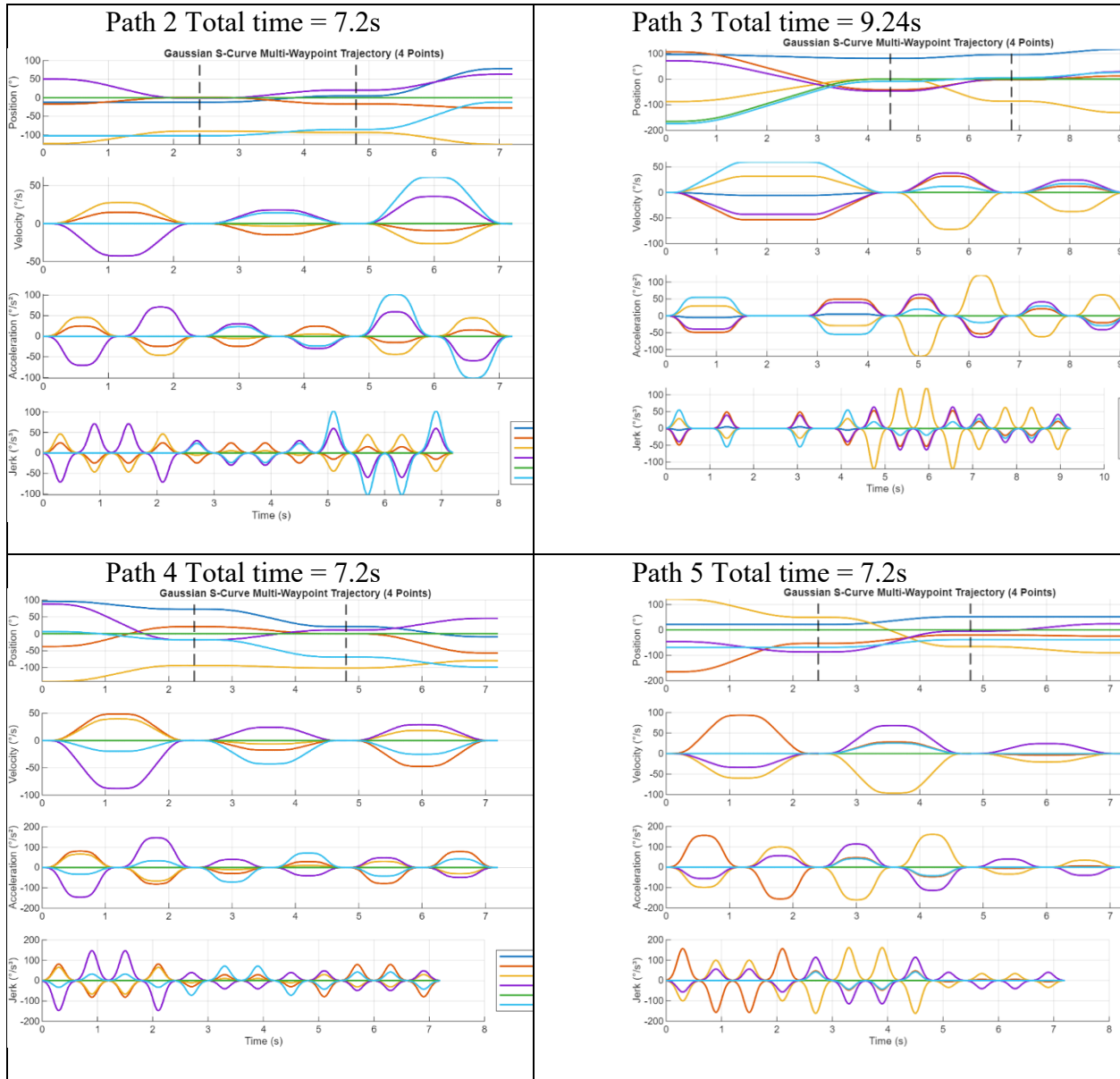


Figure 4.7 Kinematic Profile of individual joint motions for Gaussian S-Curve.

Table 4.5 Kinematic profiles for Paths 2 to 5



5. CONCLUSION AND FUTURE WORK

5.1 Summary of Contributions

The results indicate that the 6-7-6 polynomial is superior for real-time applications due to its low computational algebraic cost, whereas the erf-based S-curve is superior for extremely sensitive mechanical payloads where maximum vibration suppression is required, both achieving time-optimality due to the superior W-MPSO algorithm. The primary aim of the thesis was to deal with the inherent clash between time-optimal robotic execution and mechanical longevity, a roadblock outlined by the need to understate the travel time without destructive high-frequency vibration triggered by boundless jerk. When the classical kinematic planner works, it abstracts the robot links as massless geometric shapes. This thesis undertakes trajectory planning, guaranteeing that the generated profile respects the operator's body torque limits and finds the optimal time of execution. The techniques were modelled and validated over the 6-DOF Elephant Robotics MyCobot 280 Pi.

As an alternative for global polynomial and the computational complexity of piecewise spline (NURBS), the thesis proposes two new trajectory planning paradigms driven by a robust metaheuristic optimization engine:

The 6-7-6 polynomial formulation: For acceleration, intermediate, and deceleration stages, respectively, a new piecewise analytic trajectory has been developed using polynomial degrees 6, 7 and 6. The method achieves strict C^4 continuity (continuous catch and differentiable jerk), which does not require iterative numeric integration. The analytic nature of this formulation allows for computationally negligible trajectory coevals, which makes it very feasible for real-time management architectures.

Error function (*erf*)-based S-curve: Advancing beyond classical constant-jerk S-curves, this research introduced a probabilistic-inspired kinematic model using the Gaussian distribution function. By defining the jerk profile as a series of parameterised Gaussian pulses and integrating via the error function, the resulting trajectory achieved infinite differentiability. This method provided distinct, smooth bell-curve jerk transitions that strictly bounded the mechanical energy.

W-MOPSO Optimization Framework: A weighted multi-objective particle swarm optimization (W-MOPSO) algorithm was engineered to navigate the highly non-linear kinematics

of the 6-DOF manipulator. The algorithm successfully decoupled the spatial path geometry from the temporal execution law. By heavily penalizing Cartesian orientation errors and bounding the search space against strict velocity, acceleration, and torque constraints, the W-MOPSO engine converged on the absolute minimum execution time that remained dynamically feasible.

5.2 Key Findings

The experimental simulations and kinematic profile analyses yielded several critical insights into advanced motion planning:

Elimination of Jerk Spikes: The 6-7-6 trajectory eliminates the impulsive, jerk spike characteristic of the cubic and quintic planners. The result is a derivative profile that demonstrates that the C^4 boundary successfully translates into a smooth, uninterrupted third derivative gesture.

Superior Time-Scaling via Gaussian Pulses: The *erf*-based S-curve demonstrated superior performance in finding the balance between the quick motion and smooth motion. The isolated Gaussian jerk pulses injected the minimum possible high-frequency vibration into the joints, proving that statistical functions can be highly effective when mapped to temporal kinematics. Both 6-7-6 and the Gaussian S-curve function successfully bound the jerk value.

5.3 Future Work

This thesis proposes trajectory planning algorithms that have demonstrated significant theoretical and simulated success. Nonetheless, much can still be done for roboticists to realise better robotic motion through these algorithms. Below are the suggestions for future exploration and refinement:

Hardware implementation: In this endeavour, the algorithms were designed and tested within the limits of a desktop-sized collaborative robot. Moving forward, it would be beneficial to apply the 6-7-6 and *erf*-based planner to large industrial operators, such as industrial KUKA or FANUC manipulators. Additionally, it would be beneficial to look at results from accelerometers of moving around heavy loads.

Dynamic obstacle avoidance: The optimization framework of this thesis is based on an unchanging environment, with predefined waypoints. By incorporating real-time sensor feedback

to adjust the W-MOPSO fitness function, the trajectory could detect and define these waypoints better respond to moving obstacles while maintaining smoothness and Gaussian jerk properties.

Explicit energy optimization: Since minimization of jerk inherently reduces mechanical wear and power spikes, a dedicated energy consumption model could be integrated into the fitness analysis of the W-MOPSO algorithm. By calculating the exact electrical power required for the computed torsion, the planner could produce a strictly energy-optimal trajectory for a battery-powered robot.

Machine learning integration: Exploring the utilization of deep reinforcement learning (DRL) to instantly approximate the optimum duration breaks (a and b) for the Gaussian pulsation could bypass the iterative driven method wholly, pushing the erf-based S-curve into the kingdom of instantaneous, microsecond-level adaptive control.

REFERENCES

1. J. J. Craig, P. Prentice, and P. P. Hall, *Introduction to Robotics Mechanics and Control Third Edition*, 3rd ed. Pearson Education, Inc., 2005.
2. O. A. Detesan and I. F. Moholea, “Unimate and Beyond: Exploring the Genesis of Industrial Robotics,” *Mechanisms and Machine Science*, vol. 157 MMS, pp. 263–271, 2024, doi:10.1007/978-3-031-59257-7_27.
3. S. B. Niku, *Introduction to robotics: analysis, control, applications*. Wiley, 2011.
4. Chuck Lewin, “Mathematics of Motion Control Profiles,” Performance Motion Devices Articles. Accessed: Oct. 23, 2025. [Online]. Available: <https://www.pmdcorp.com/resources/type/articles/get/mathematics-of-motion-control-profiles-article>
5. W. Cai, Y. Gan, B. Zhou, and X. Dai, “Smooth and Time-optimal Improved-Bezier Trajectory Planning Based on Dynamics for Industrial Robots.”
6. A. Lunia *et al.*, “Modeling, Motion Planning, and Control of Manipulators and Mobile Robots,” Mar. 21, 2023. Accessed: Apr. 25, 2026. [Online]. Available: <https://opentextbooks.clemson.edu/wangrobotics/>
7. W. Wang, Q. Tao, Y. Cao, X. Wang, and X. Zhang, “Robot Time-Optimal Trajectory Planning Based on Improved Cuckoo Search Algorithm,” *IEEE Access*, vol. 8, pp. 86923–86933, 2020, doi: 10.1109/ACCESS.2020.2992640.
8. “Trajectory Generation – Modeling, Motion Planning, and Control of Manipulators and Mobile Robots.” Accessed: Oct. 23, 2025. [Online]. Available: <https://opentextbooks.clemson.edu/wangrobotics/chapter/trajectory-generation/>
9. F. Wang, Z. Wu, and T. Bao, “Time-Jerk optimal Trajectory Planning of Industrial Robots Based on a Hybrid WOA-GA Algorithm,” *Processes*, vol. 10, no. 5, May 2022, doi:10.3390/pr10051014.
10. A. Gasparetto and V. Zanotto, “A new method for smooth trajectory planning of robot manipulators,” *Mech. Mach. Theory*, vol. 42, no. 4, pp. 455–471, Apr. 2007, doi:10.1016/J.MECHMACHTHEORY.2006.04.002.
11. J. E. Lee, A. Bylard, R. Sun, and L. Sentis, “On the Performance of Jerk-Constrained Time-Optimal Trajectory Planning for Industrial Manipulators,” in *Proceedings - IEEE International Conference on Robotics and Automation*, Institute of Electrical and Electronics Engineers Inc., 2024, pp. 9772–9778. doi:10.1109/ICRA57147.2024.10610437.

12. G. Wu and S. Zhang, "Real-time jerk-minimization trajectory planning of robotic arm based on polynomial curve optimization," *Proc. Inst. Mech. Eng. C J. Mech. Eng. Sci.*, vol. 236, no. 21, pp. 10852–10864, Nov. 2022, doi: 10.1177/09544062221106632.
13. L. Zhang, Y. Li, and Y. Zhang, "Trajectory planning of seven-degree-of-freedom redundant manipulator in narrow space," *AIP Adv.*, vol. 14, no. 12, Dec. 2024, doi:10.1063/5.0231168.
14. J. Zhao, X. Zhu, and T. Song, "Serial Manipulator Time-Jerk Optimal Trajectory Planning Based on Hybrid IWOA-PSO Algorithm," *IEEE Access*, vol. 10, pp. 6592–6604, 2022, doi: 10.1109/ACCESS.2022.3141448.
15. X. He and Y. Zhou, "Time-optimal Trajectory Planning for Robotic Manipulator," in *2025 IEEE 20th Conference on Industrial Electronics and Applications, ICIEA 2025*, Institute of Electrical and Electronics Engineers Inc., 2025. doi:10.1109/ICIEA65512.2025.11149228.
16. W. Wang, Q. Tao, Y. Cao, X. Wang, and X. Zhang, "Robot Time-Optimal Trajectory Planning Based on Improved Cuckoo Search Algorithm," *IEEE Access*, vol. 8, pp. 86923–86933, 2020, doi: 10.1109/ACCESS.2020.2992640.
17. J. E. Lee, A. Bylard, R. Sun, and L. Sentis, "On the Performance of Jerk-Constrained Time-Optimal Trajectory Planning for Industrial Manipulators," in *Proceedings - IEEE International Conference on Robotics and Automation*, Institute of Electrical and Electronics Engineers Inc., 2024, pp. 9772–9778. doi:10.1109/ICRA57147.2024.10610437.
18. C. Qian, J. Yao, and Y. Zhang, "Time-Jerk Optimal Robotic Trajectory Planning Under Jerk and Continuity Constraints via Convex Optimization," *Actuators*, vol. 14, no. 6, Jun. 2025, doi: 10.3390/act14060272.
19. G. Braglia, M. Tagliavini, F. Pini, and L. Biagiotti, "Online Motion Planning for Safe Human–Robot Cooperation Using B-Splines and Hidden Markov Models," *Robotics*, vol. 12, no. 4, Aug. 2023, doi: 10.3390/robotics12040118.
20. Andreas Aristidou and Joan Lasembly, "Inverse Kinematics: a review of existing techniques and introduction of a new fast iterative solver." Accessed: Oct. 29, 2025. [Online]. Available: https://www.researchgate.net/profile/Andreas-Aristidou/publication/273166356_Inverse_Kinematics_a_review_of_existing_techniques_and_introduction_of_a_new_fast_iterative_solver/links/54faeca10cf20b0d2cb8782b/Inverse-Kinematics-a-review-of-existing-techniques-and-introduction-of-a-new-fast-iterative-solver.pdf
21. "Introduction · GitBook." Accessed: Nov. 02, 2025. [Online]. Available: https://docs.elephantrobotics.com/docs/mycobot_280_pi_en/
22. "ER myCobot 280 PI 2023 - Elephant Robotics." Accessed: Oct. 26, 2025. [Online]. Available: <https://www.elephantrobotics.com/en/mycobot-280-pi-2023-en/>
23. "Universal Robots - Cobots are Making a Difference." Accessed: Apr. 25, 2026. [Online]. Available: <https://blog.knottscs.com/blog/universal-robots-cobots-are-making-a-difference>

24. S. Luo, D. Chu, Q. Li, and Y. He, "Inverse Kinematics Solution of 6-DOF Manipulator Based on Multi-Objective Full-Parameter Optimization PSO Algorithm," *Front. Neurorobot.*, vol. 16, Mar. 2022, doi: 10.3389/fnbot.2022.791796.
25. N. Rokbani, B. Neji, M. Slim, S. Mirjalili, and R. Ghandour, "A Multi-Objective Modified PSO for Inverse Kinematics of a 5-DOF Robotic Arm," *Applied Sciences (Switzerland)*, vol. 12, no. 14, Jul. 2022, doi: 10.3390/app12147091.
26. R. Katzschmann, T. Kroger, T. Asfour, and O. Khatib, "Towards online trajectory generation considering robot dynamics and torque limits," *IEEE International Conference on Intelligent Robots and Systems*, pp. 5644–5651, 2013, doi:10.1109/IROS.2013.6697174.
27. K. M. Lynch and F. C. Park, "MODERN ROBOTICS MECHANICS, PLANNING, AND CONTROL," 2017, Accessed: Feb. 16, 2026. [Online]. Available: <http://modernrobotics.org.Commentsarewelcome!>
28. M. W. . Spong, Seth. Hutchinson, and M. . Vidyasagar, "Robot modeling and control," 2020, Accessed: Feb. 16, 2026. [Online]. Available: <https://www.wiley.com/en-us/Robot+Modeling+and+Control%2C+2nd+Edition-p-9781119524045>
29. C. Yang *et al.*, "Time-Jerk Optimal Trajectory Planning of Robot based on Dung Beetle Optimizer Algorithm," in *7th International Conference on Intelligent Robotics and Control Engineering, IRCE 2024*, Institute of Electrical and Electronics Engineers Inc., 2024, pp. 100–105. doi:10.1109/IRCE62232.2024.10739825.
30. R. Shadmehr, S. P. Wise, and N. Hogan, "Supplementary documents for 'Computational Neurobiology of Reaching and Pointing', by A MINIMUM-JERK TRAJECTORY".
31. S. Zhao, T. Zheng, C. Wang, S. Zhao, Y. Zhu, and J. Zhao, "Interaction-Friendly Trajectories via Torque-and-Jerk-Constrained Optimization," *Lecture Notes in Computer Science*, vol. 16075 LNCS, pp. 589–600, 2026, doi:10.1007/978-981-95-2098-5_50.
32. Z. Fan *et al.*, "A Cartesian-Based Trajectory Optimization with Jerk Constraints for a Robot," *Entropy 2023, Vol. 25*, vol. 25, no. 4, Apr. 2023, doi:10.3390/e25040610.
33. L. Piegl and W. Tiller, "Rational B-spline Curves and Surfaces," pp. 117–139, 1997, doi: 10.1007/978-3-642-59223-2_4.
34. P. Lambrechts, M. Boerlage, and M. Steinbuch, "Trajectory planning and feedforward design for high performance motion systems," pp. 4637–4642 vol.5, May 2018, doi:10.23919/acc.2004.1384042.
35. T. Chinyai, L. Tan, and M. Yao, "Time-Optimal Trajectory Planning for Robotic Manipulators with Differentiable Jerk Minimization," pp. 0150–0155, Dec. 2025, doi:10.1109/UEMCON67449.2025.11267653.

36. X. Zhou, T. Chinyai, and L. Tan, "Multi-Objective Optimization for Inverse Kinematics of Robotic Arm with Joint Configuration," in *Conference Proceedings - IEEE SOUTHEASTCON*, Institute of Electrical and Electronics Engineers Inc., 2025, pp. 514–519. doi:10.1109/SoutheastCon56624.2025.10971491.
37. C. Ji, Z. Zhang, G. Cheng, M. Kong, and R. Li, "A Convex Optimization Method to Time-Optimal Trajectory Planning with Jerk Constraint for Industrial Robotic Manipulators," *IEEE Transactions on Automation Science and Engineering*, vol. 21, no. 4, pp. 7629–7646, 2024, doi:10.1109/TASE.2023.3346693.
38. N. Wu, D. Jia, Z. Li, and Z. He, "Trajectory Planning of Robotic Arm Based on Particle Swarm Optimization Algorithm," *Applied Sciences (Switzerland)*, vol. 14, no. 18, Sep. 2024, doi:10.3390/app14188234.
39. G. Wu and N. Zhang, "Kinematically Constrained Jerk–Continuous S-Curve Trajectory Planning in Joint Space for Industrial Robots," *Electronics (Switzerland)*, vol. 12, no. 5, Mar. 2023, doi:10.3390/electronics12051135.
40. R. Bussola, G. Incerti, C. Remino, and M. Tiboni, "S-Curve Trajectory Planning for Industrial Robots Based on Curvature Radius," *Robotics*, vol. 14, no. 11, Nov. 2025, doi:10.3390/robotics14110155.

PUBLICATIONS

1. T. Chinyai, L. Tan, and M. Yao, “Time-Optimal Trajectory Planning for Robotic Manipulators with Differentiable Jerk Minimization,” pp. 0150–0155, Dec. 2025.
2. X. Zhou, T. Chinyai, and L. Tan, “Multi-Objective Optimization for Inverse Kinematics of Robotic Arm with Joint Configuration,” in Conference Proceedings - IEEE SOUTHEASTCON, Institute of Electrical and Electronics Engineers Inc., 2025, pp. 514–519.